deeplearning.ai

Case Studies

Why look at case studies?

Outline

Classic networks:

- LeNet-5 ←
- AlexNet ←
- VGG ←

ResNet (152)

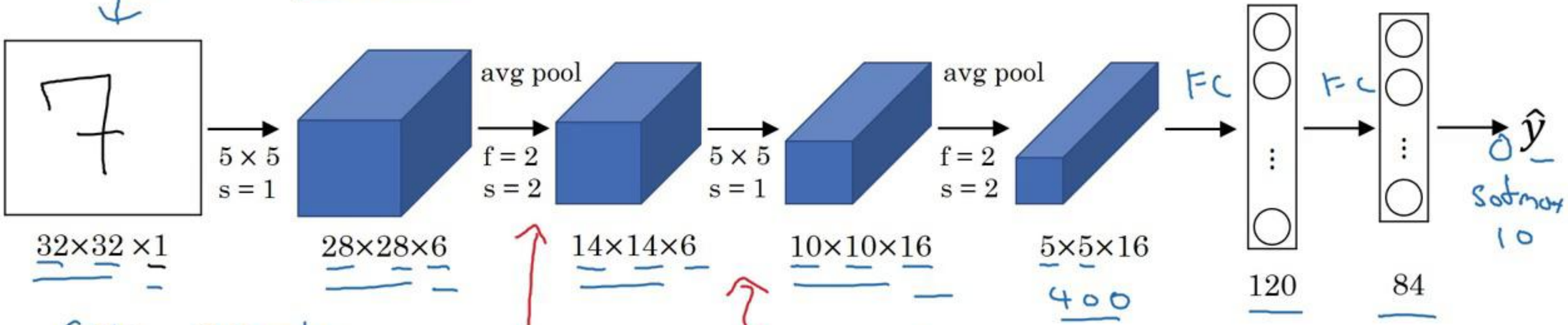
Inception

deeplearning.ai

Case Studies

Classic networks

LeNet - 5



60K parameters.

non-linearity after pooling

$n_H \times n_W \times n_C$ $f \times f \times n_C$

$n_H, n_W \downarrow$ $n_C \uparrow$

conv pool conv pool fc fc output

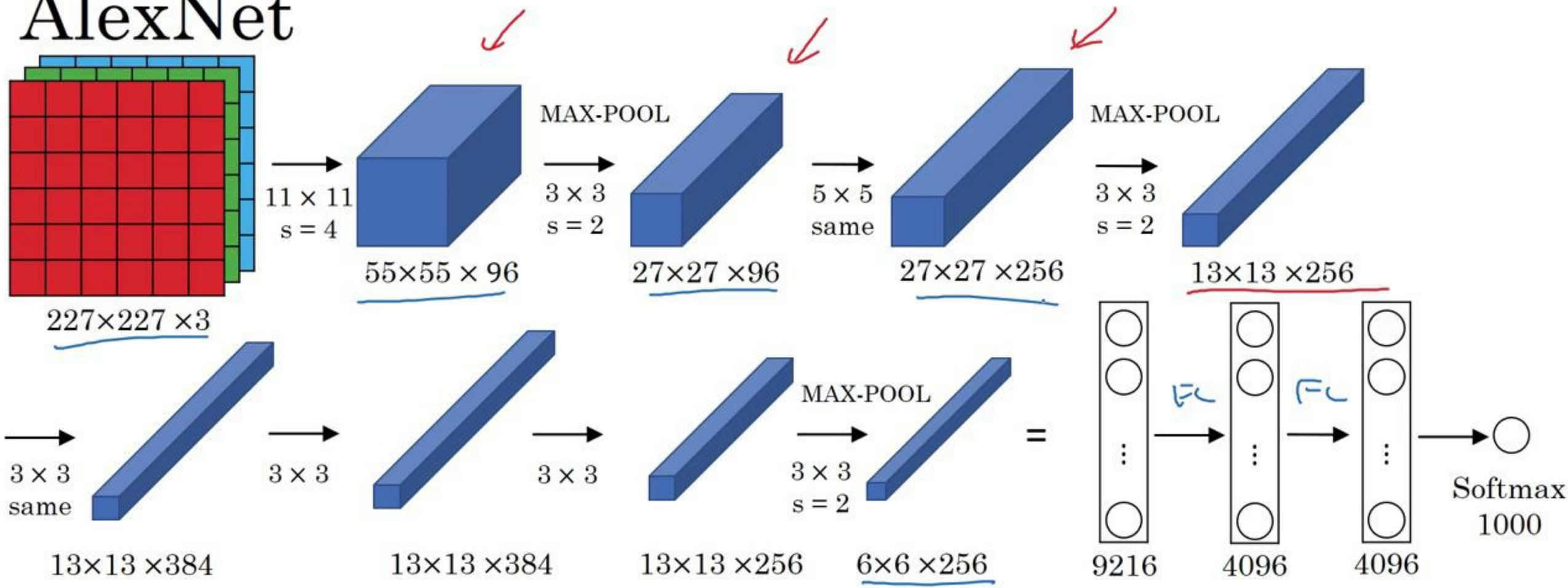
Advanced: Sigmoid/tanh ReLU

II III

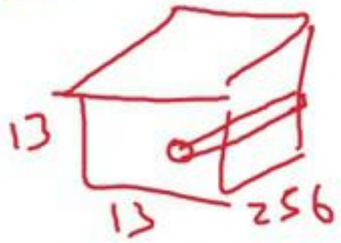
[LeCun et al., 1998. Gradient-based learning applied to document recognition]

Andrew Ng

AlexNet



- Similar to LeNet, but much bigger. 9216 ~60M parameters
- ReLU
- Multiple GPUs.
- Local Response Normalization (LRN)



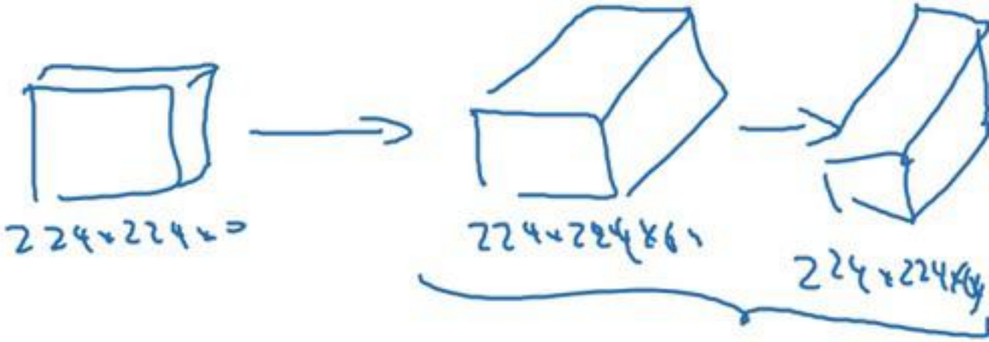
[Krizhevsky et al., 2012. ImageNet classification with deep convolutional neural networks]

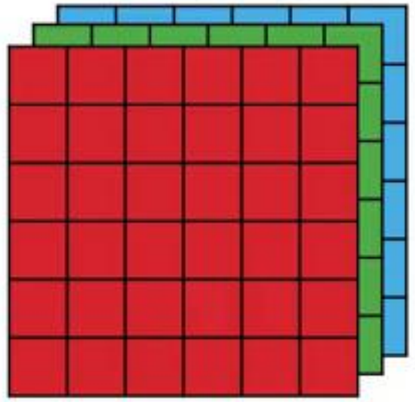
Andrew Ng

VGG - 16

VGG-19

CONV = 3×3 filter, s = 1, same MAX-POOL = 2×2 , s = 2





224×224 ×3

→ [CONV 64] ×2 → 224×224×64 → POOL → 112×112 ×64 → [CONV 128] ×2 → 112×112 ×128 → POOL → 56×56 ×128

→ [CONV 256] ×3 → 56×56 ×256 → POOL → 28×28 ×256 → [CONV 512] × 3 → 28×28 ×512 → POOL → 14×14×512

→ [CONV 512] ×3 → 14×14 ×512 → POOL → 7×7×512 → FC 4096 → FC 4096 → Softmax 1000

$n_H, n_W \downarrow$ $n_C \uparrow$ ~138M

[Simonyan & Zisserman 2015. Very deep convolutional networks for large-scale image recognition]

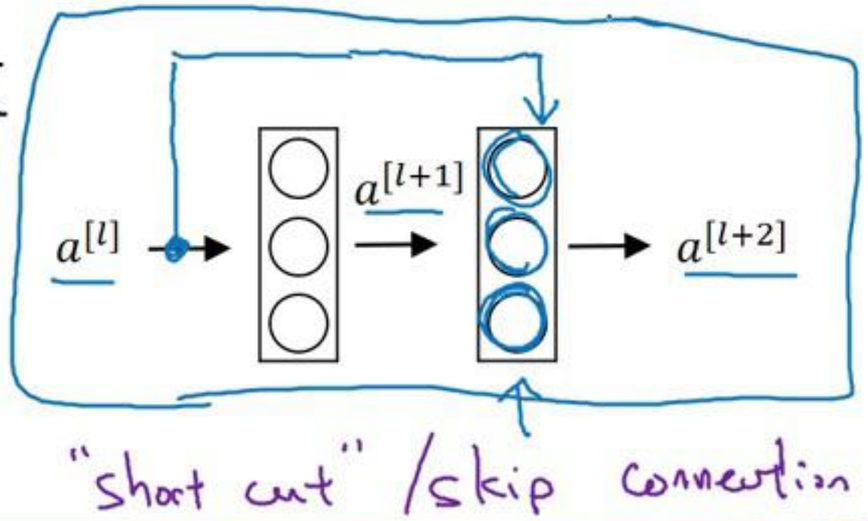
Andrew Ng

deeplearning.ai

Case Studies

Residual Networks (ResNets)

Residual block



"short cut" /skip connection

$a^{[l]} \rightarrow$ Linear $\rightarrow$ ReLU $\xrightarrow{a^{[l+1]}}$ Linear $\rightarrow \oplus \rightarrow$ ReLU $\rightarrow a^{[l+2]}$

"main path"

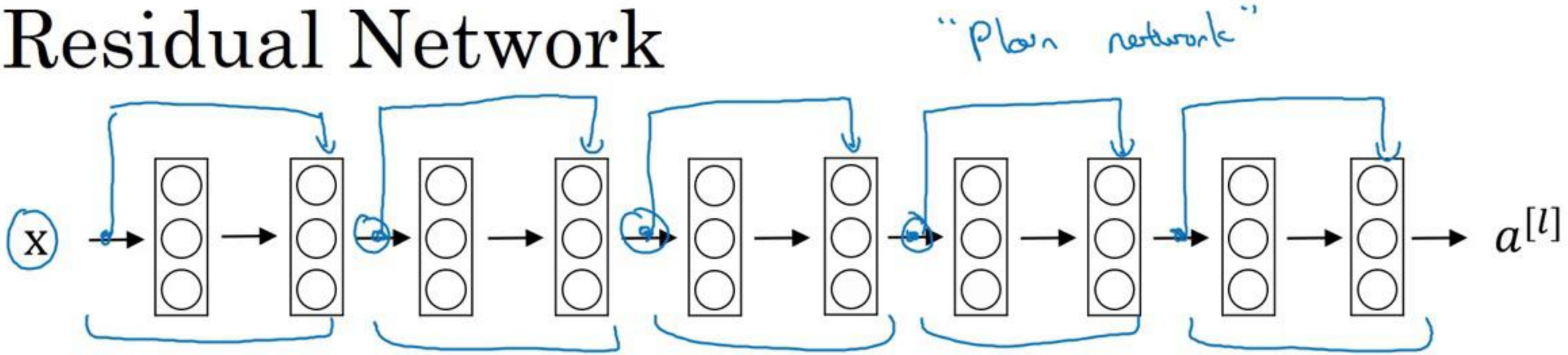
$$z^{[l+1]} = W^{[l+1]} a^{[l]} + b^{[l+1]} \qquad a^{[l+1]} = g(z^{[l+1]}) \qquad z^{[l+2]} = W^{[l+2]} a^{[l+1]} + b^{[l+2]} \qquad \cancel{a^{[l+2]} = g(z^{[l+2]})}$$

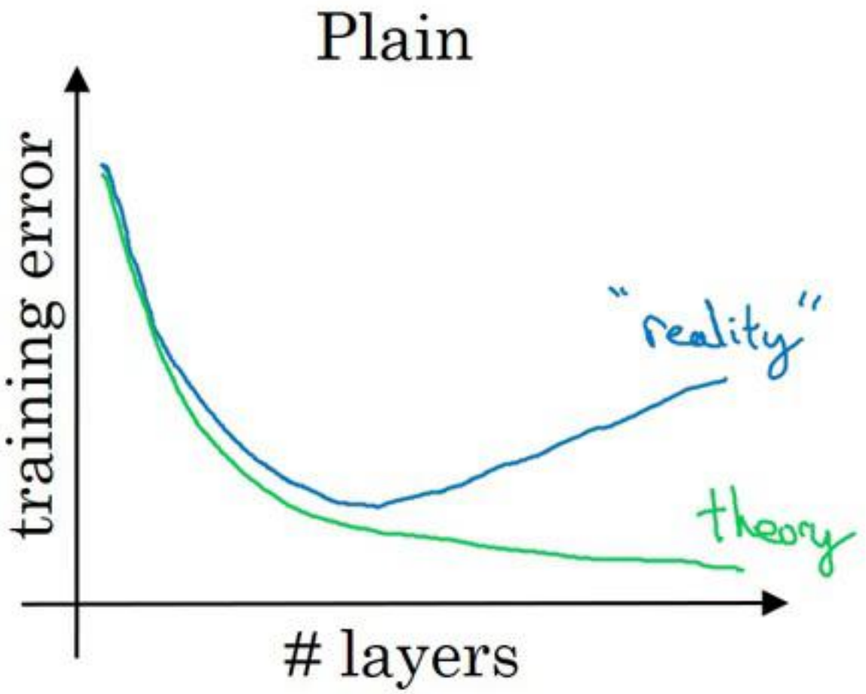
$$a^{[l+2]} = g(z^{[l+2]} + a^{[l]})$$

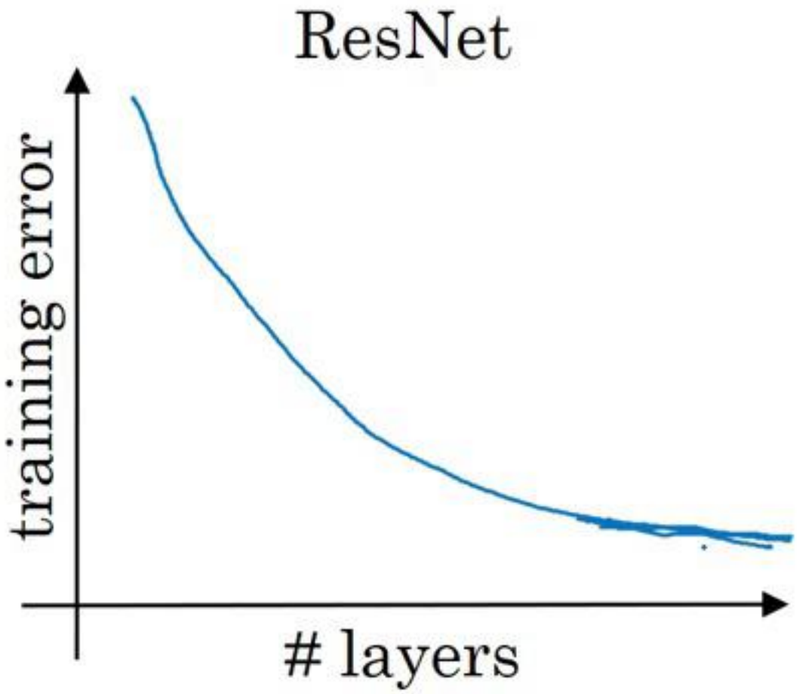
[He et al., 2015. Deep residual networks for image recognition]

Andrew Ng

Residual Network







[He et al., 2015. Deep residual networks for image recognition]

Andrew Ng

deeplearning.ai

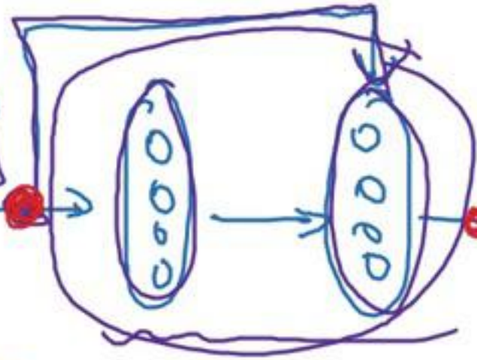
Case Studies

Why ResNets work

Why do residual networks work?

$X \rightarrow$ Big NN $\rightarrow a^{[l]}$

Identity function is easy for Residual block to learn!

$X \rightarrow$ Big NN $\rightarrow a^{[l]} \rightarrow$  $\rightarrow a^{[l+2]}$

"Same"

ReLU. $a \geq 0$

$$a^{[l+2]} = g(z^{[l+2]} + a^{[l]})$$

$$= g(\cancel{W^{[l+2]} a^{[l+1]} + b^{[l+2]}} + W_s a^{[l]}) = g(a^{[l]})$$

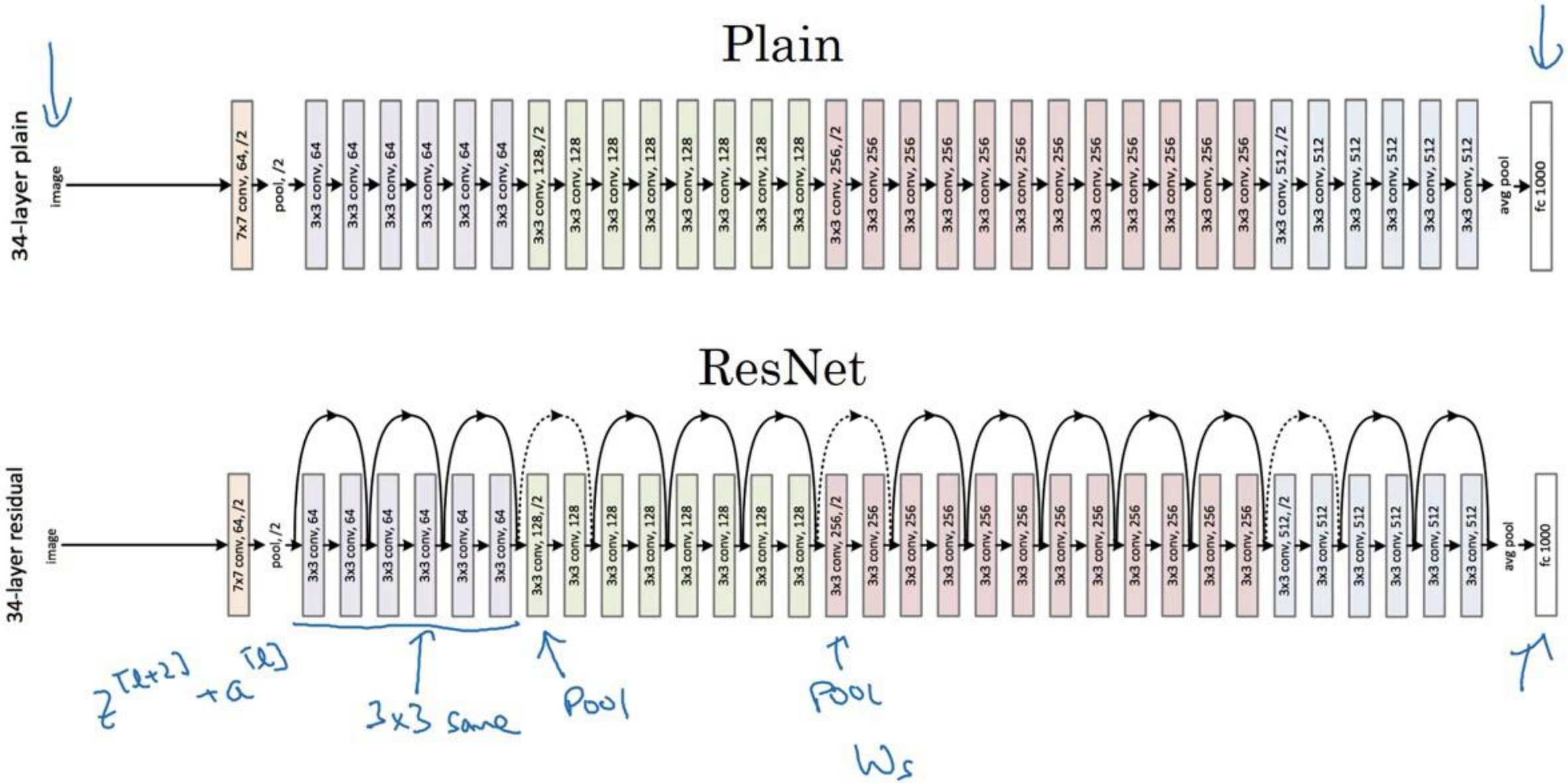
$$= a^{[l]}$$

256 $\quad$ $\mathbb{R}^{256 \times 128}$ $\quad$ 128

If $W^{[l+2]} = 0$, $b^{[l+2]} = 0$

Andrew Ng

ResNet



[He et al., 2015. Deep residual networks for image recognition]

Andrew Ng

deeplearning.ai

Case Studies

Network in Network and 1×1 convolutions

Why does a 1×1 convolution do?

1	2	3	6	5	8
3	5	5	1	3	4
2	1	3	4	9	3
4	7	8	5	7	9
1	5	3	7	4	8
5	4	9	8	3	5

$6 \times 6 \times 1$

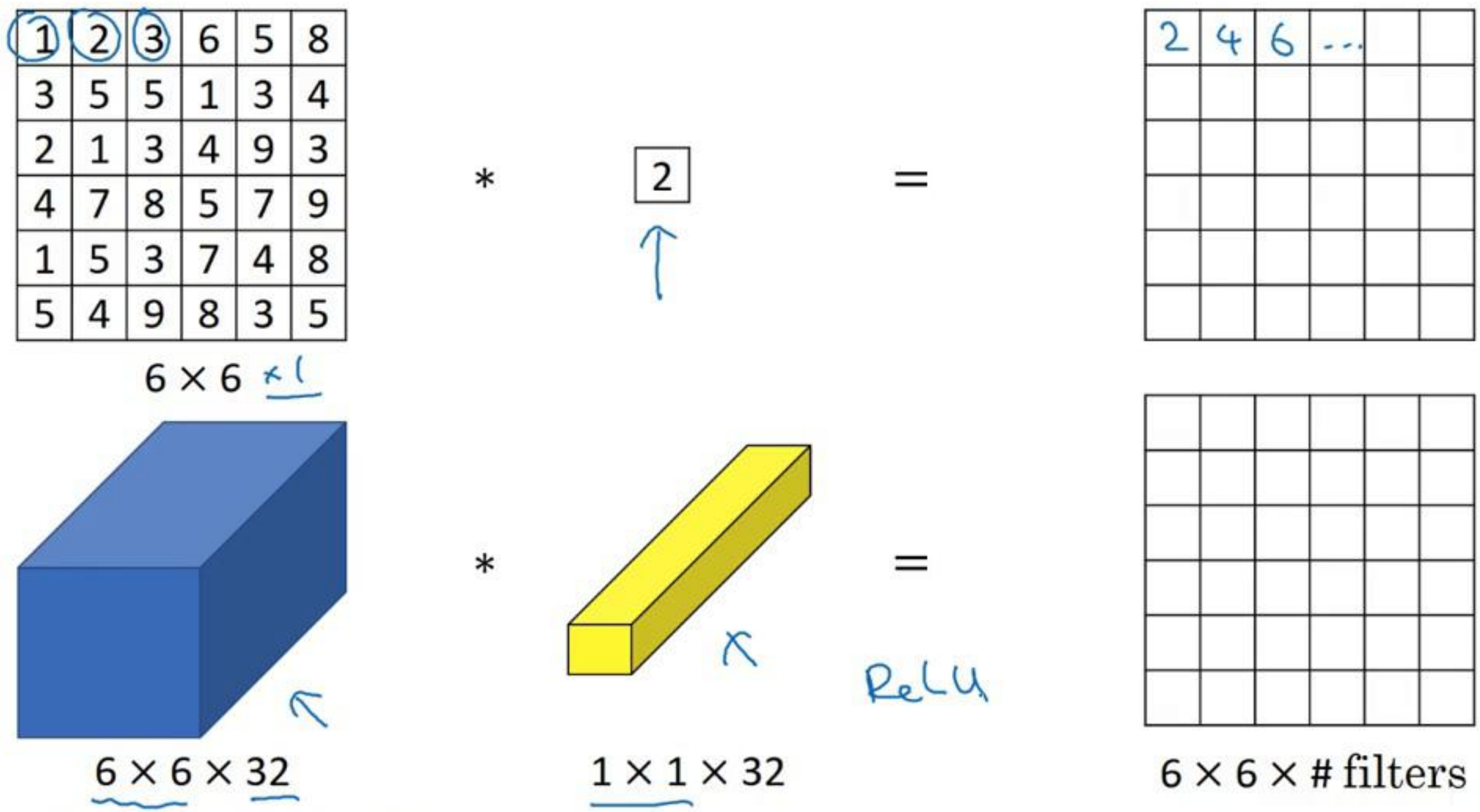
$*$ $\boxed{2}$ $=$

2	4	6	...		

[Lin et al., 2013. Network in network]

Andrew Ng

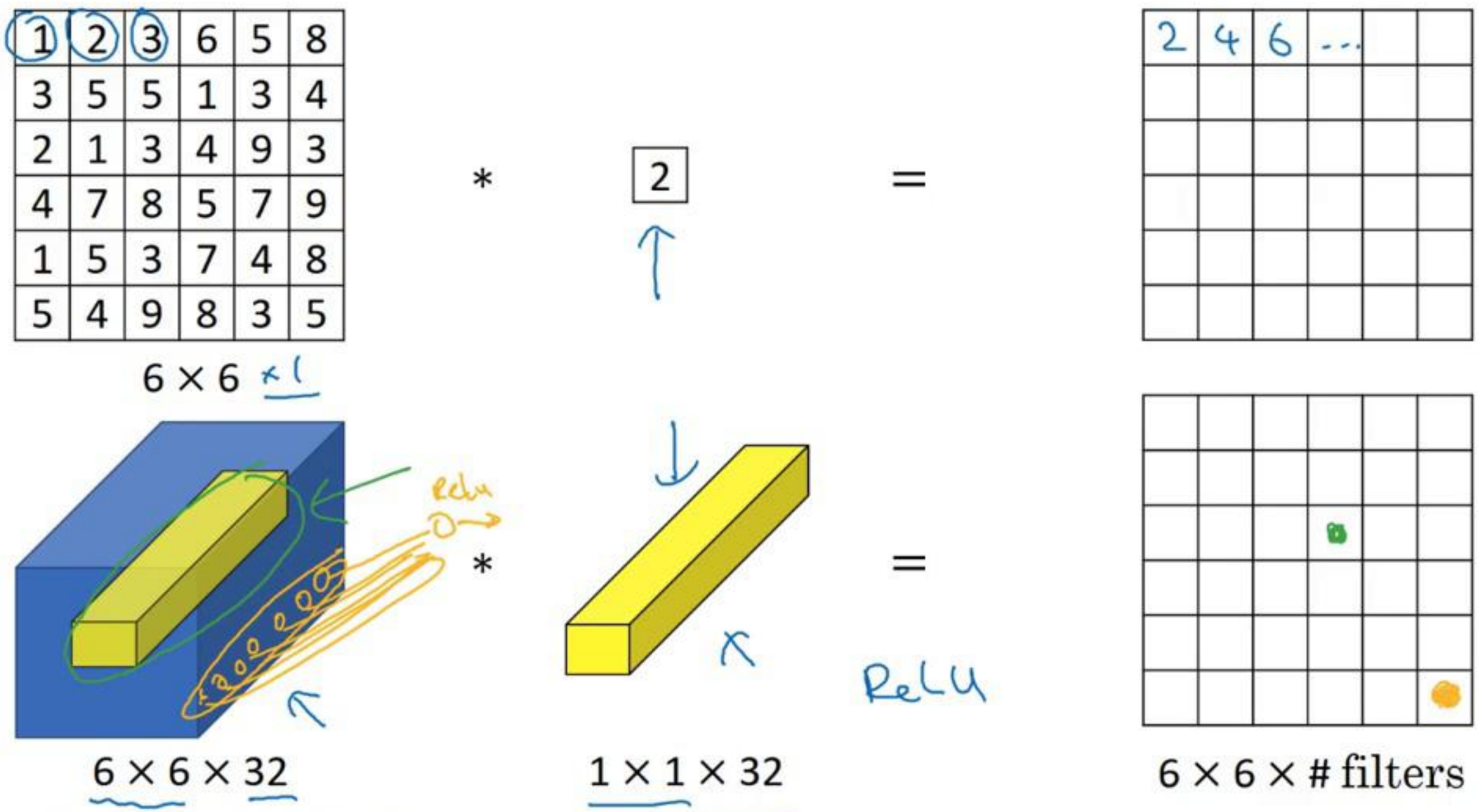
Why does a 1×1 convolution do?



[Lin et al., 2013. Network in network]

Andrew Ng

Why does a 1×1 convolution do?



[Lin et al., 2013. Network in network]

Andrew Ng

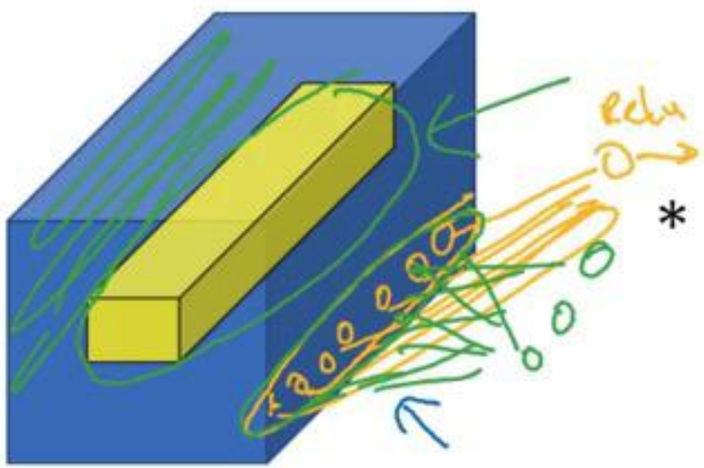
Why does a 1×1 convolution do?

1	2	3	6	5	8
3	5	5	1	3	4
2	1	3	4	9	3
4	7	8	5	7	9
1	5	3	7	4	8
5	4	9	8	3	5

$6 \times 6 \times 1$

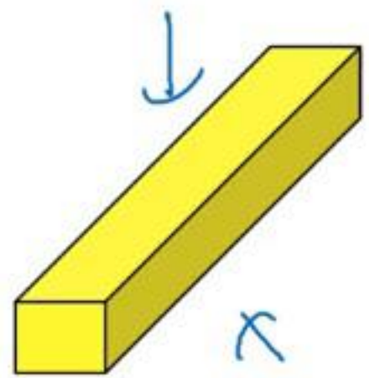
$*$ $\boxed{2}$ $=$

2	4	6	...		



$6 \times 6 \times 32$

$*$



$1 \times 1 \times 32$

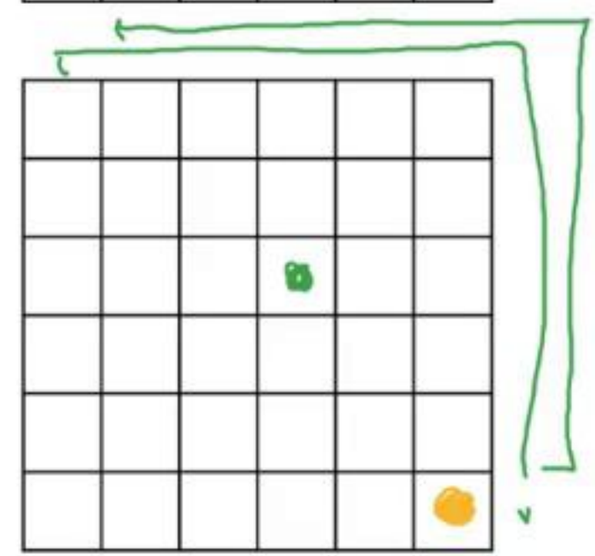
32 → # filters.

$n_c^{[l+1]}$

$=$

ReLU

Network in Network

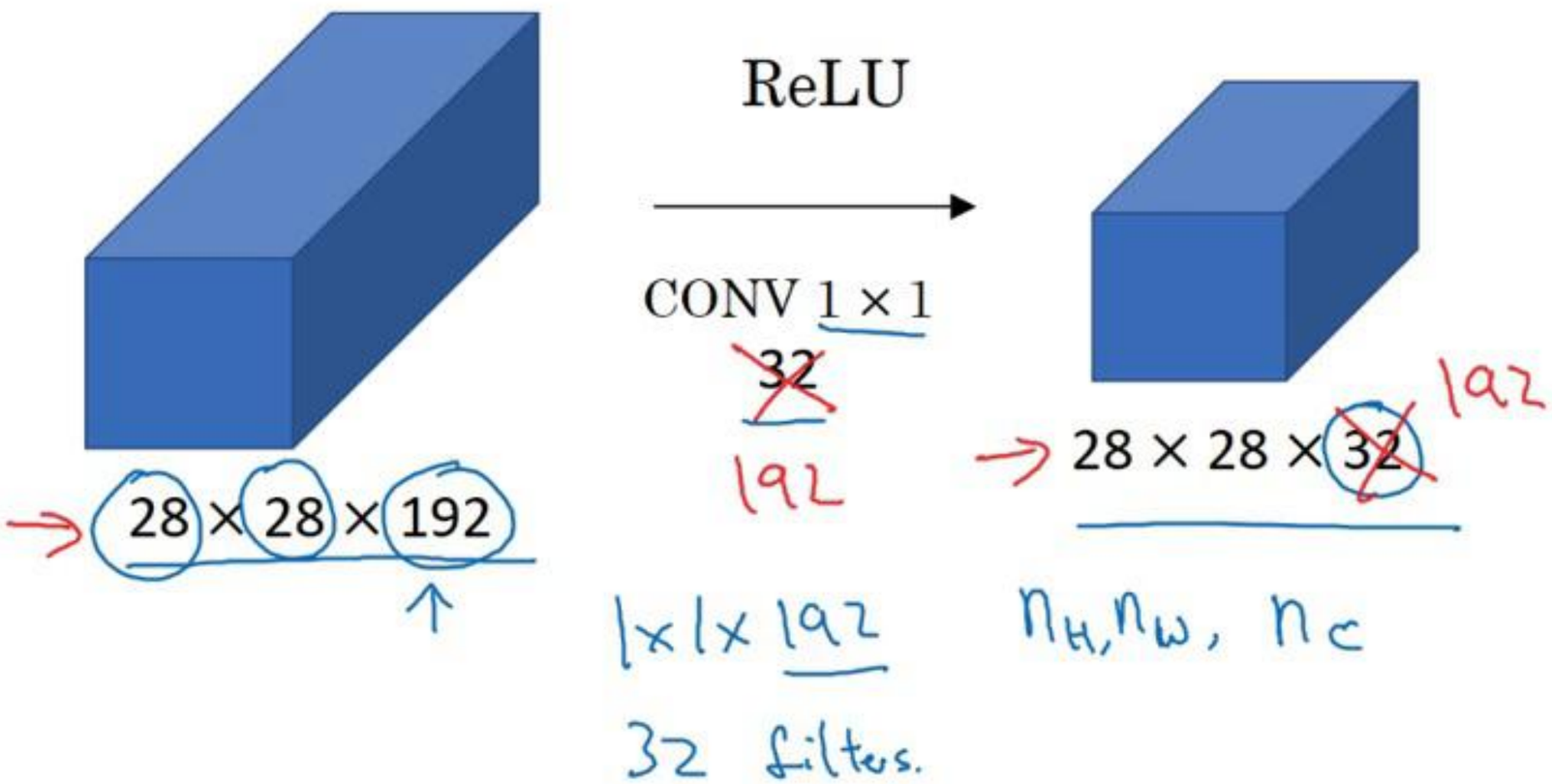


$6 \times 6 \times$ # filters

[Lin et al., 2013. Network in network]

Andrew Ng

Using 1×1 convolutions



[Lin et al., 2013. Network in network]

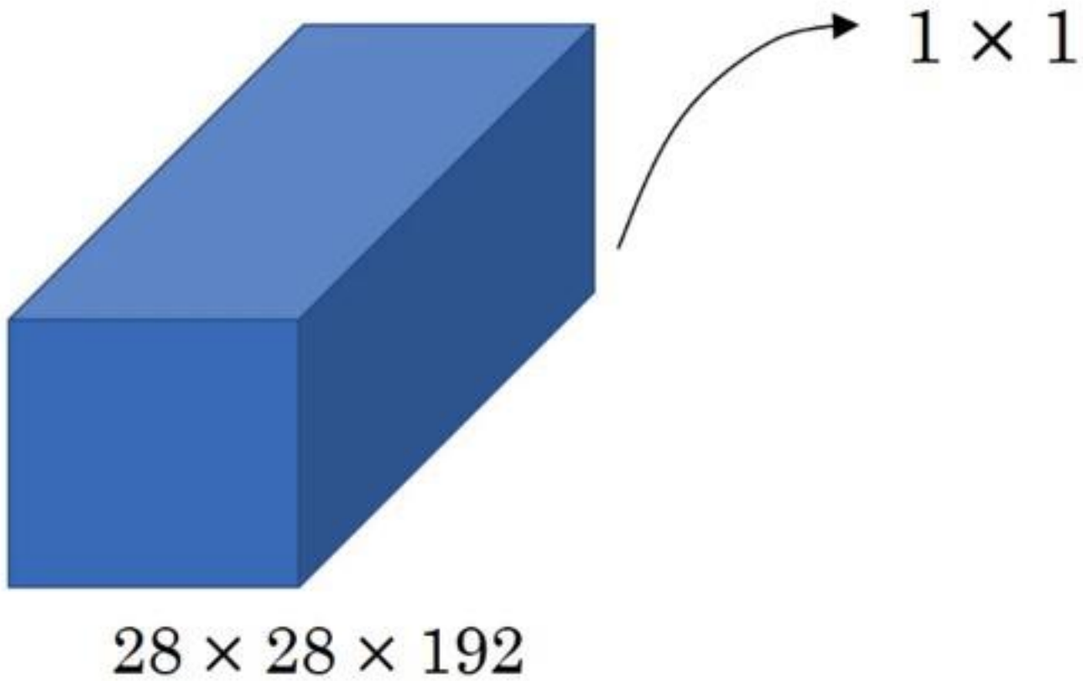
Andrew Ng

deeplearning.ai

Case Studies

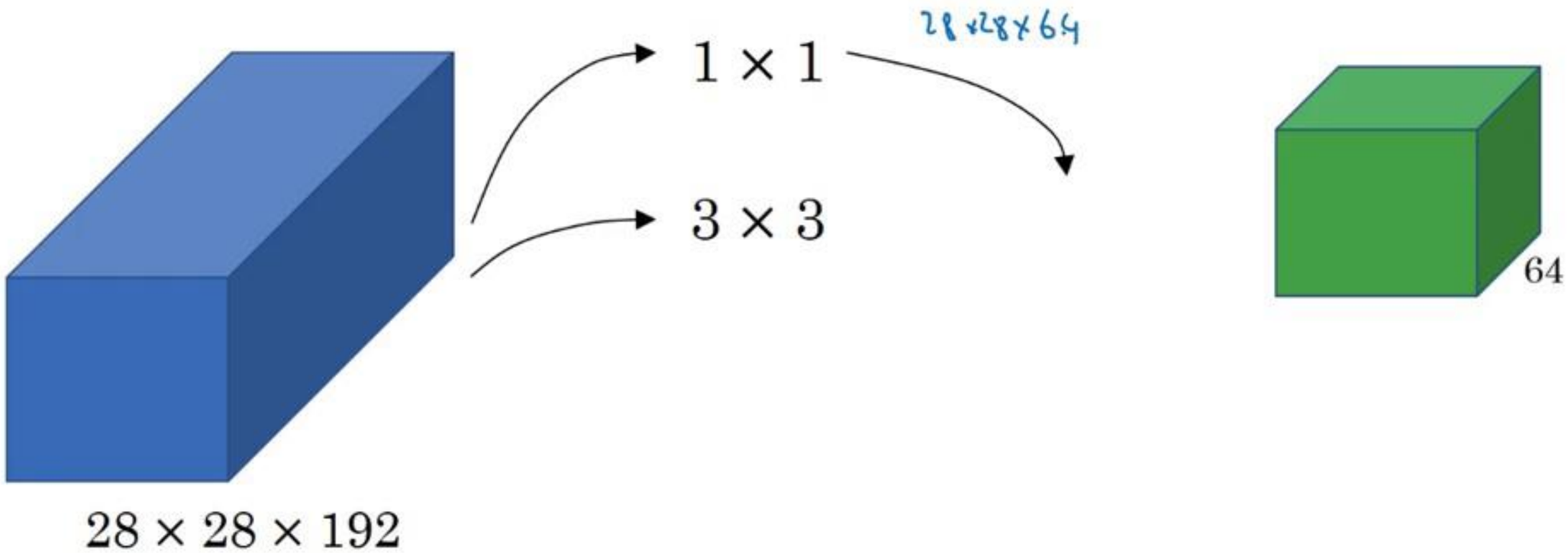
Inception network motivation

Motivation for inception network



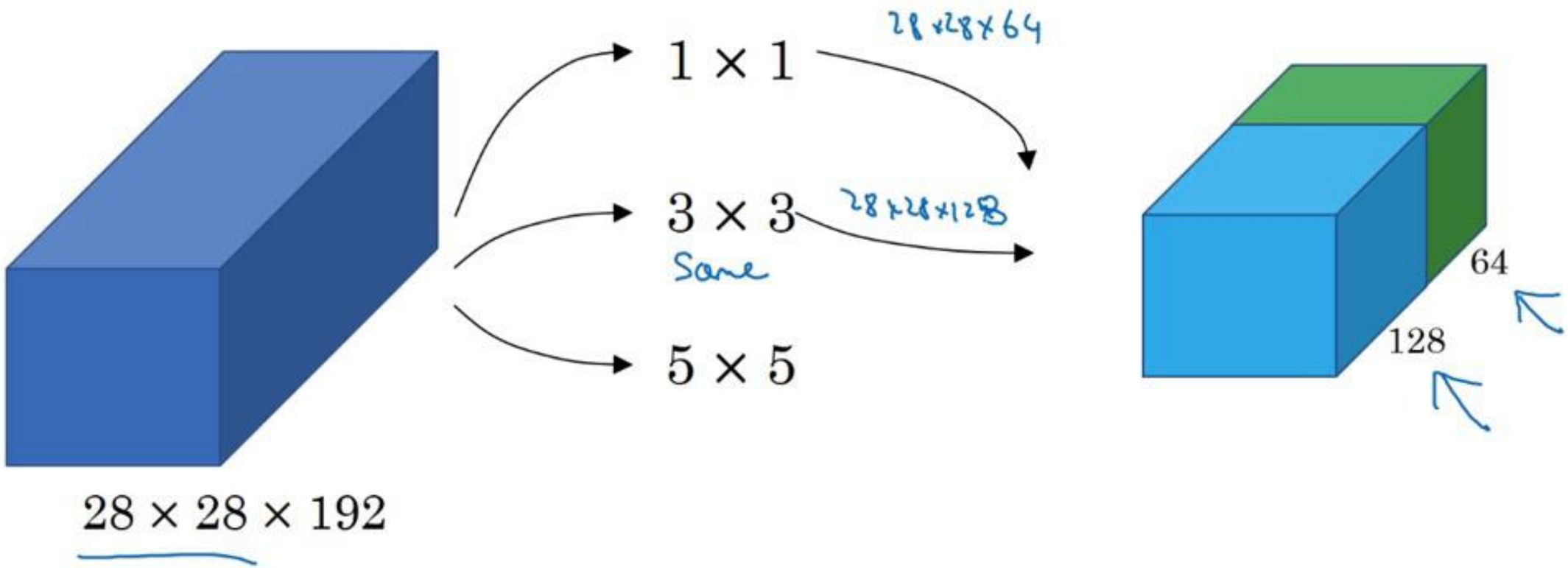
[Szegedy et al. 2014. Going deeper with convolutions]

Motivation for inception network



[Szegedy et al. 2014. Going deeper with convolutions]

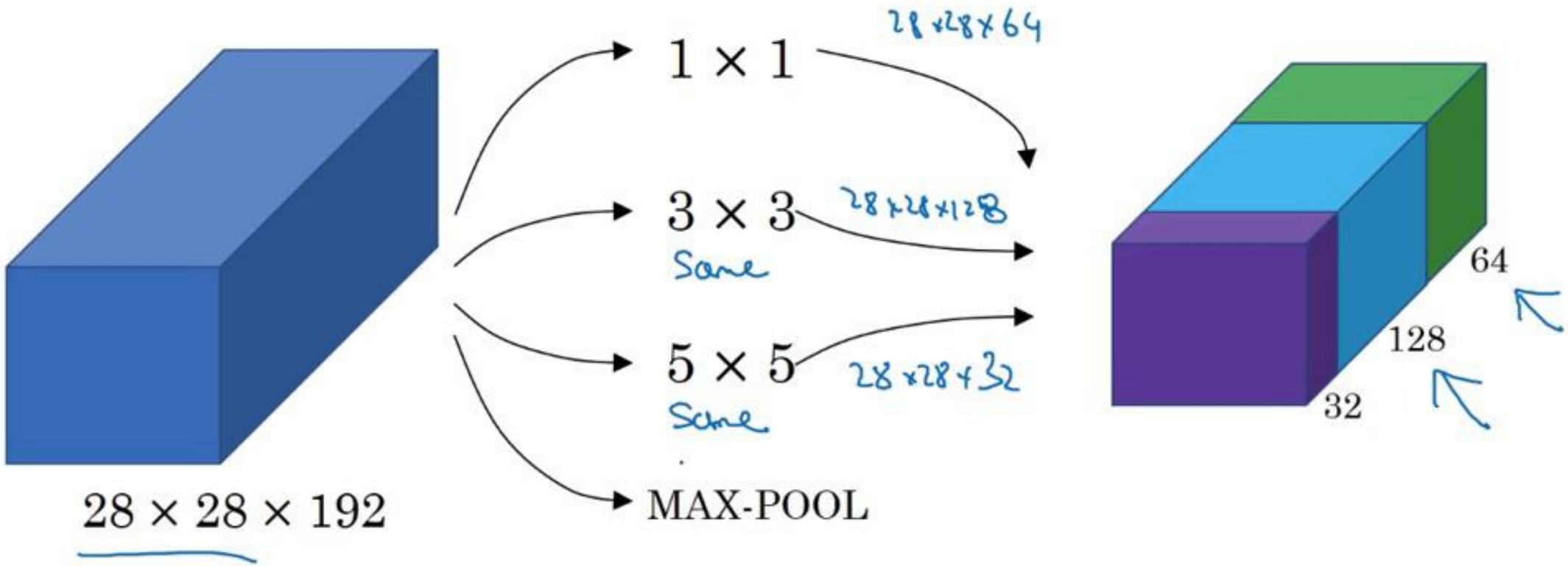
Motivation for inception network



[Szegedy et al. 2014. Going deeper with convolutions]

Andrew Ng

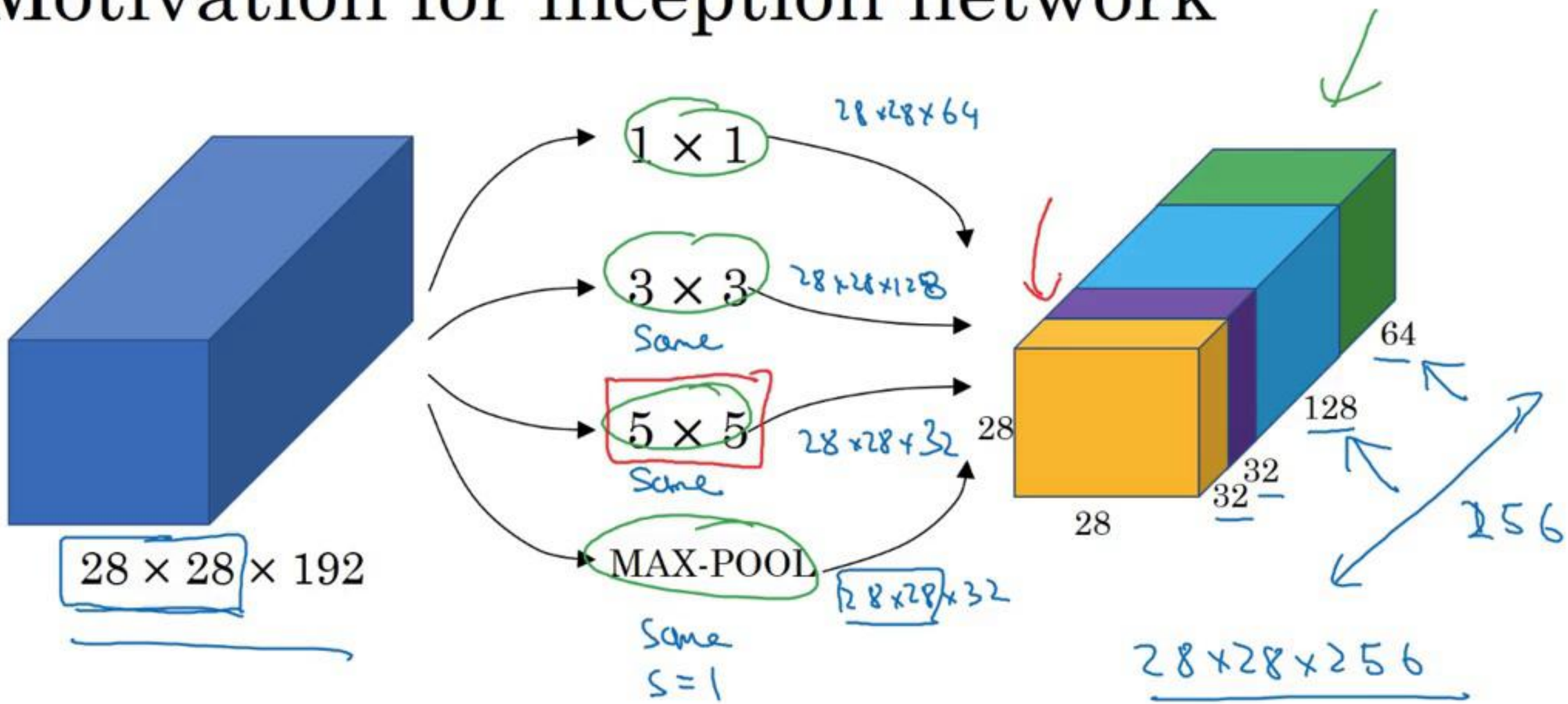
Motivation for inception network



[Szegedy et al. 2014. Going deeper with convolutions]

Andrew Ng

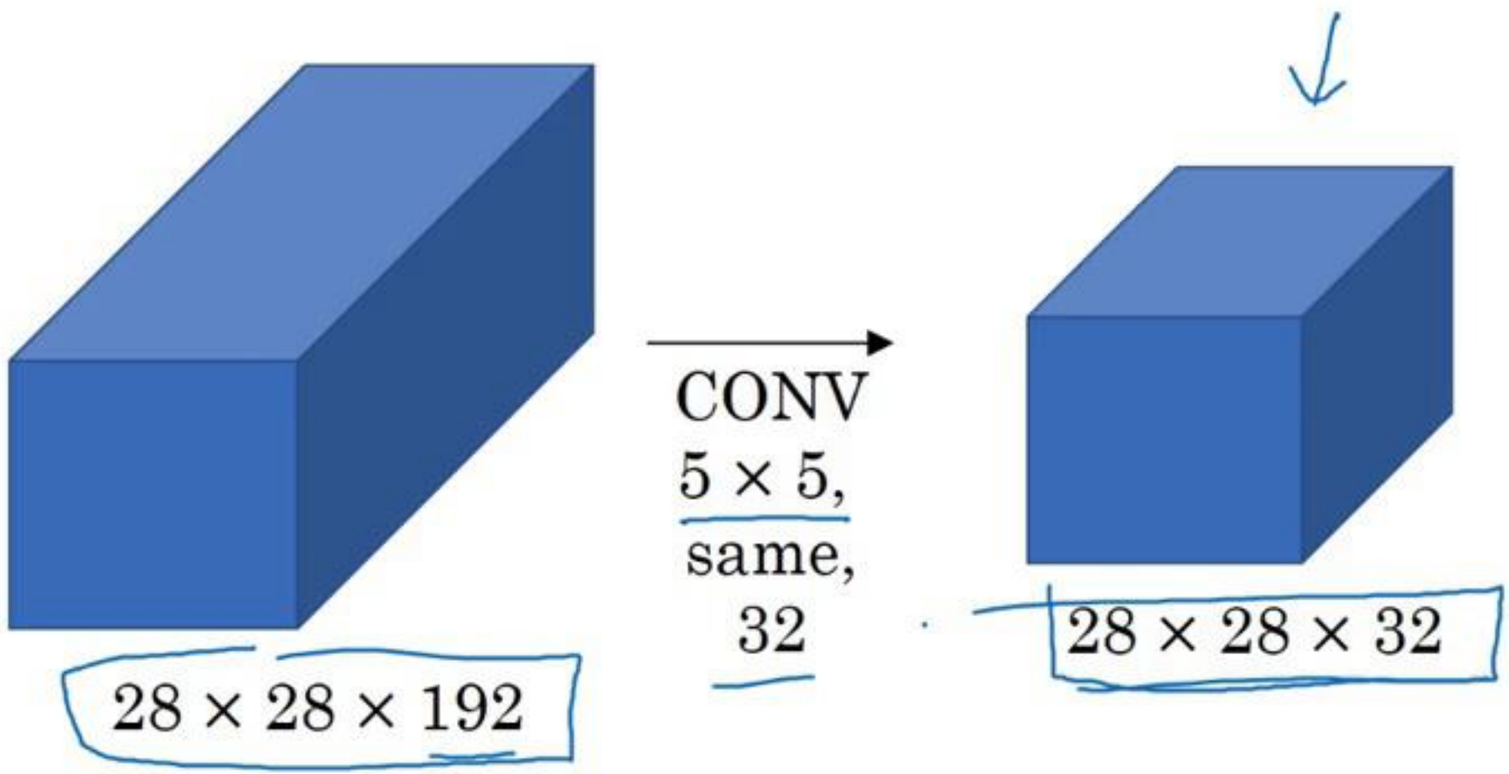
Motivation for inception network



[Szegedy et al. 2014. Going deeper with convolutions]

Andrew Ng

The problem of computational cost

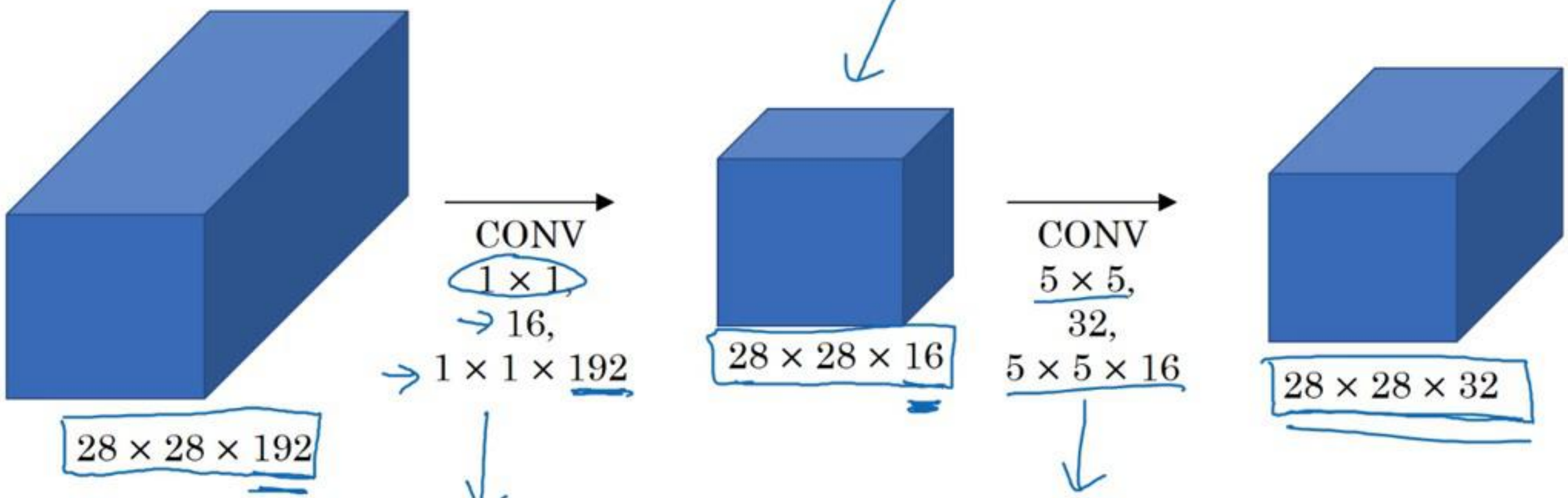


32 filters. filters are 5x5x192.

28x28x32 x 5x5x192 = 120M.

Using 1×1 convolution

"bottleneck layer"



$28 \times 28 \times 16 \times 192 = 2.4M$

$28 \times 28 \times 32 \times 5 \times 5 \times 16 = 10.0M$

$12.4M$

$120M$

Andrew Ng

deeplearning.ai

Case Studies

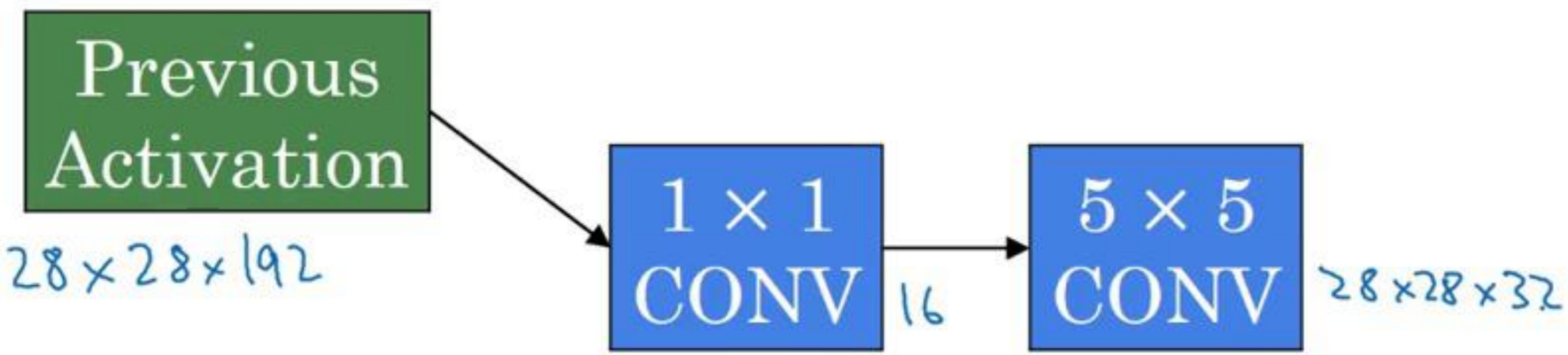
Inception network

Inception module

Previous Activation

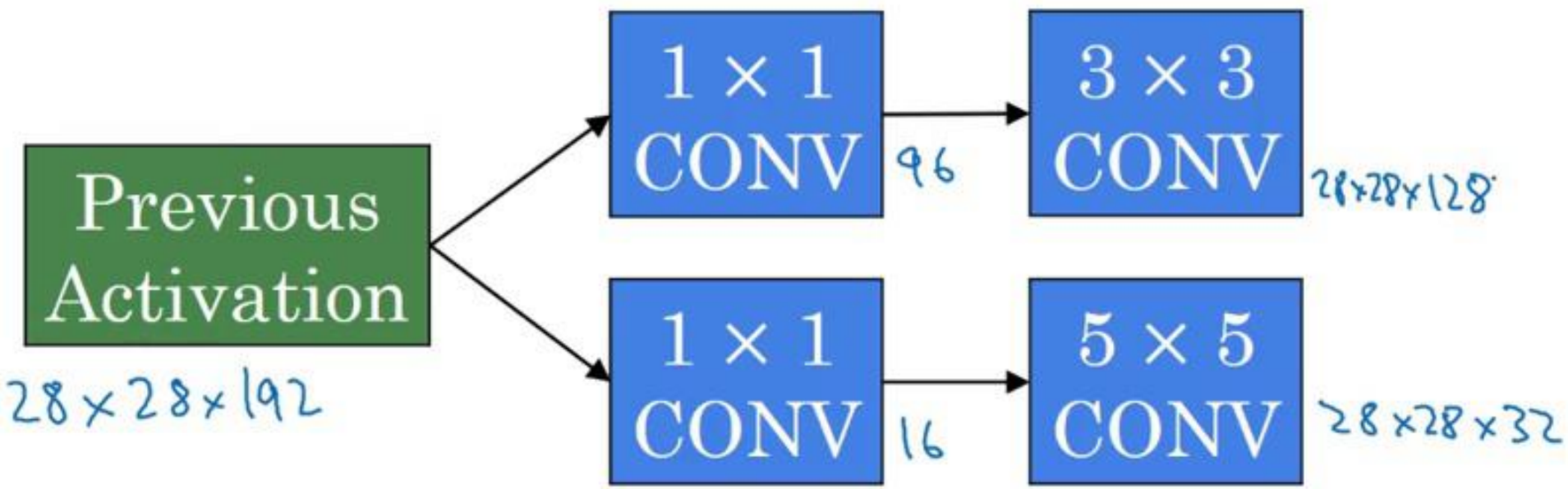
$28 \times 28 \times 192$

Inception module

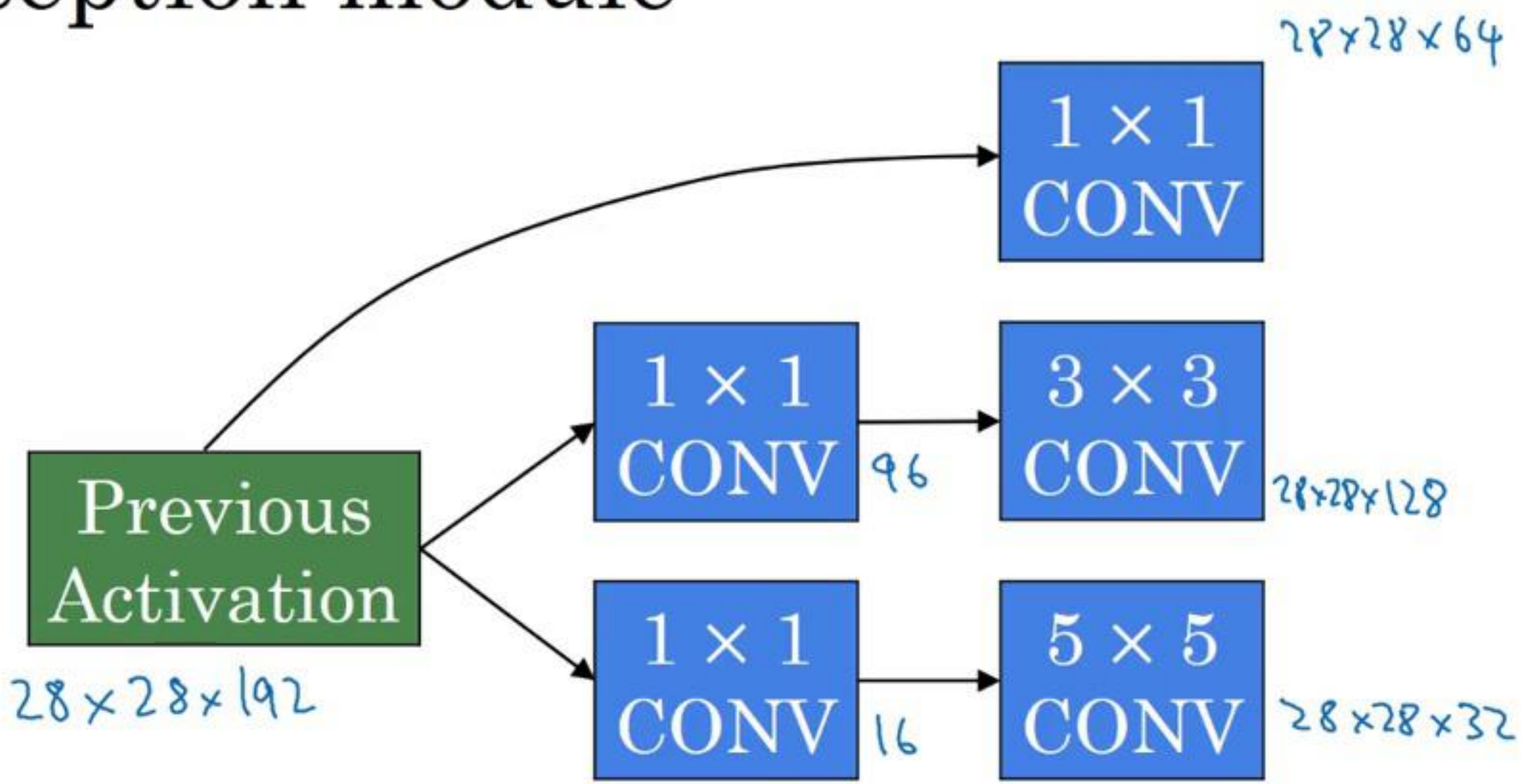


Andrew Ng

Inception module

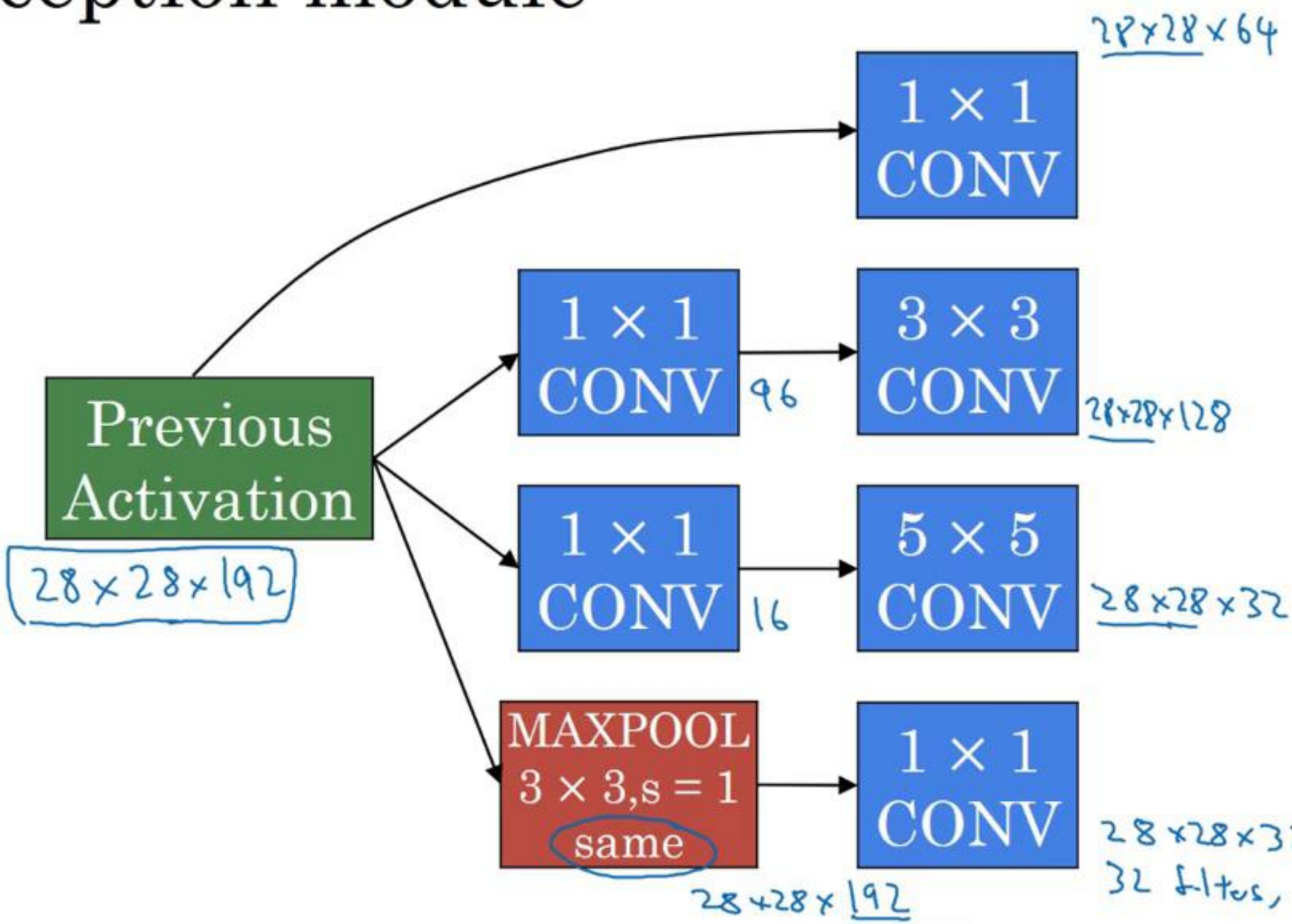


Inception module



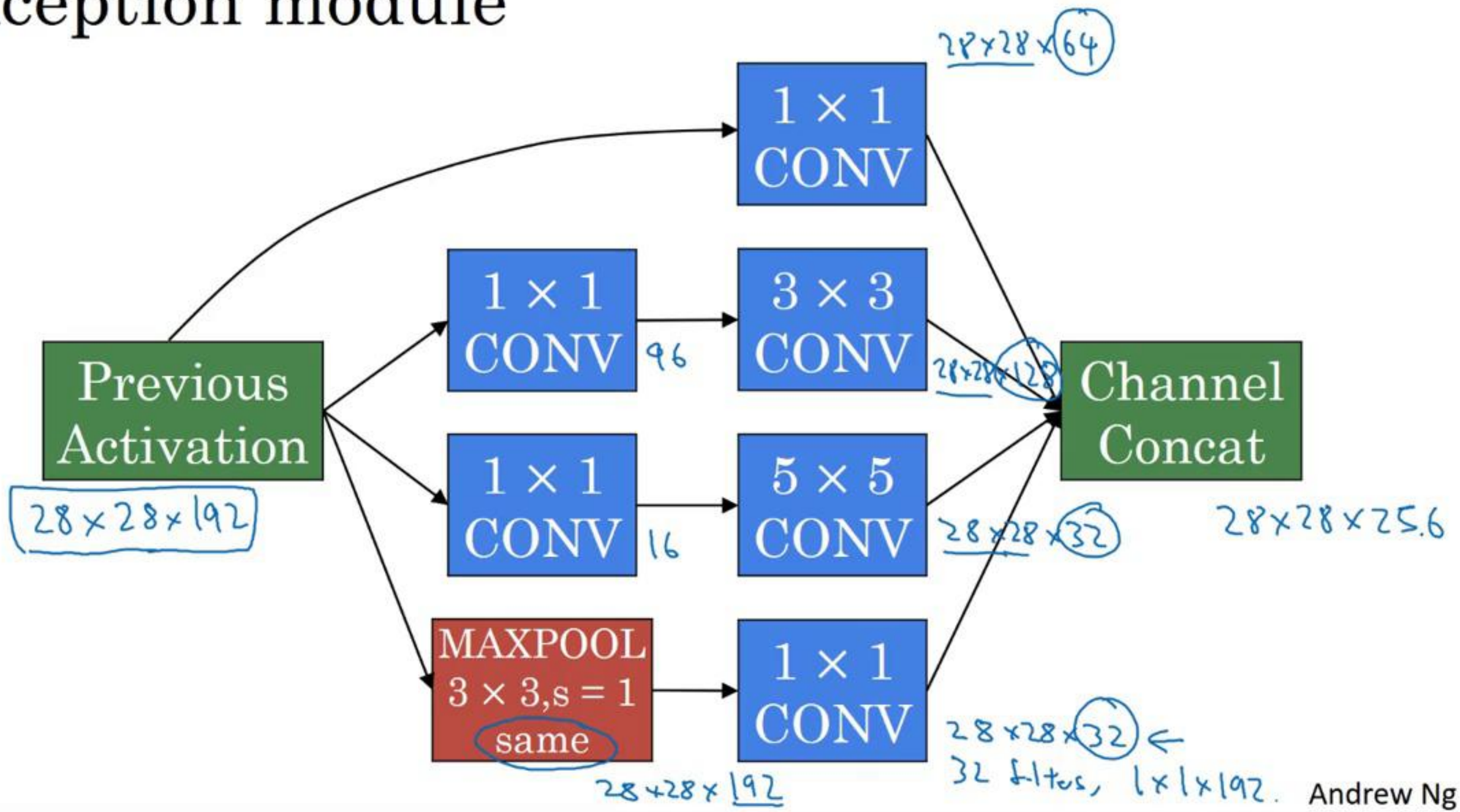
Andrew Ng

Inception module



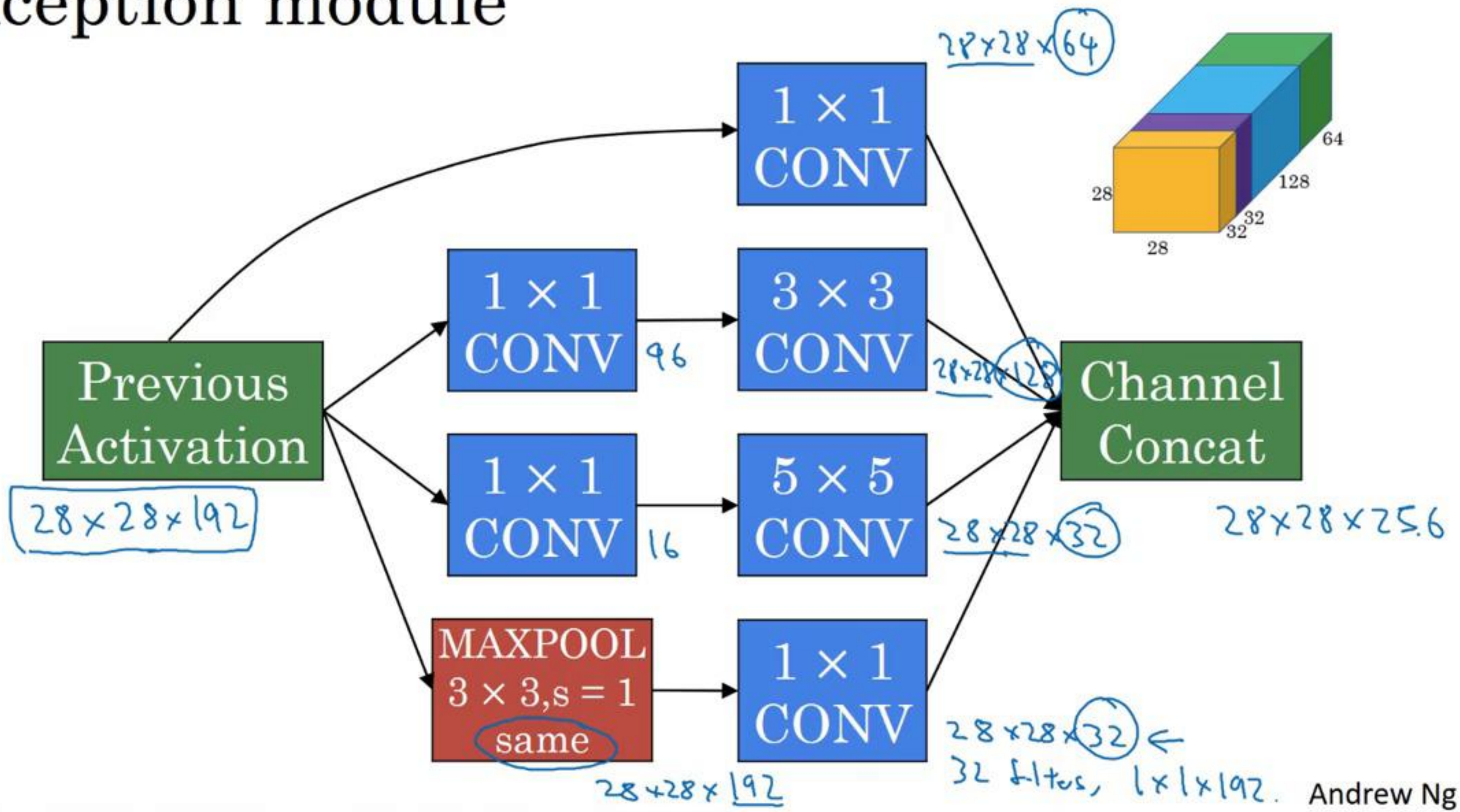
Andrew Ng

Inception module



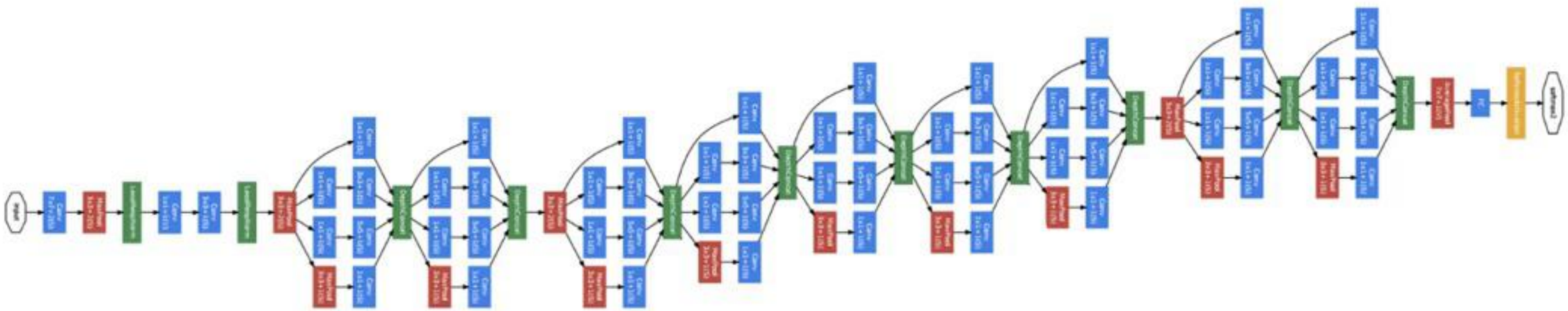
Andrew Ng

Inception module



Andrew Ng

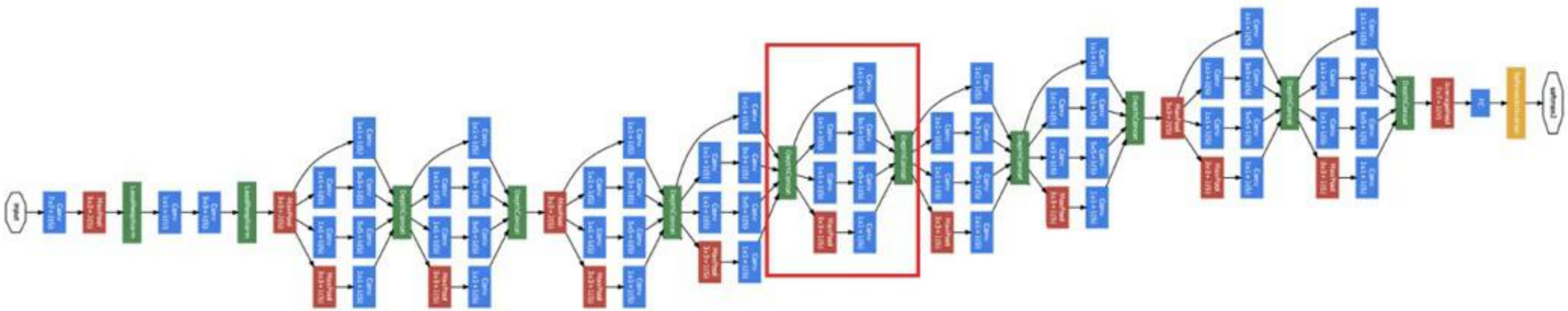
Inception network



[Szegedy et al., 2014, Going Deeper with Convolutions]

Andrew Ng

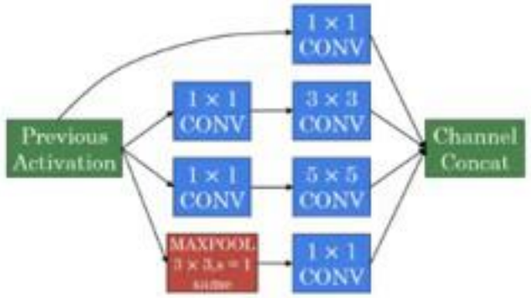
Inception network

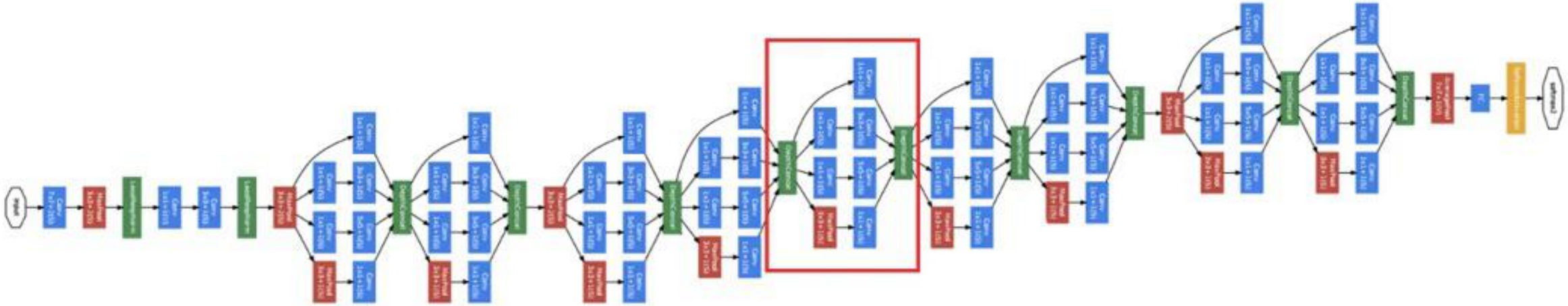


[Szegedy et al., 2014, Going Deeper with Convolutions]

Andrew Ng

Inception network

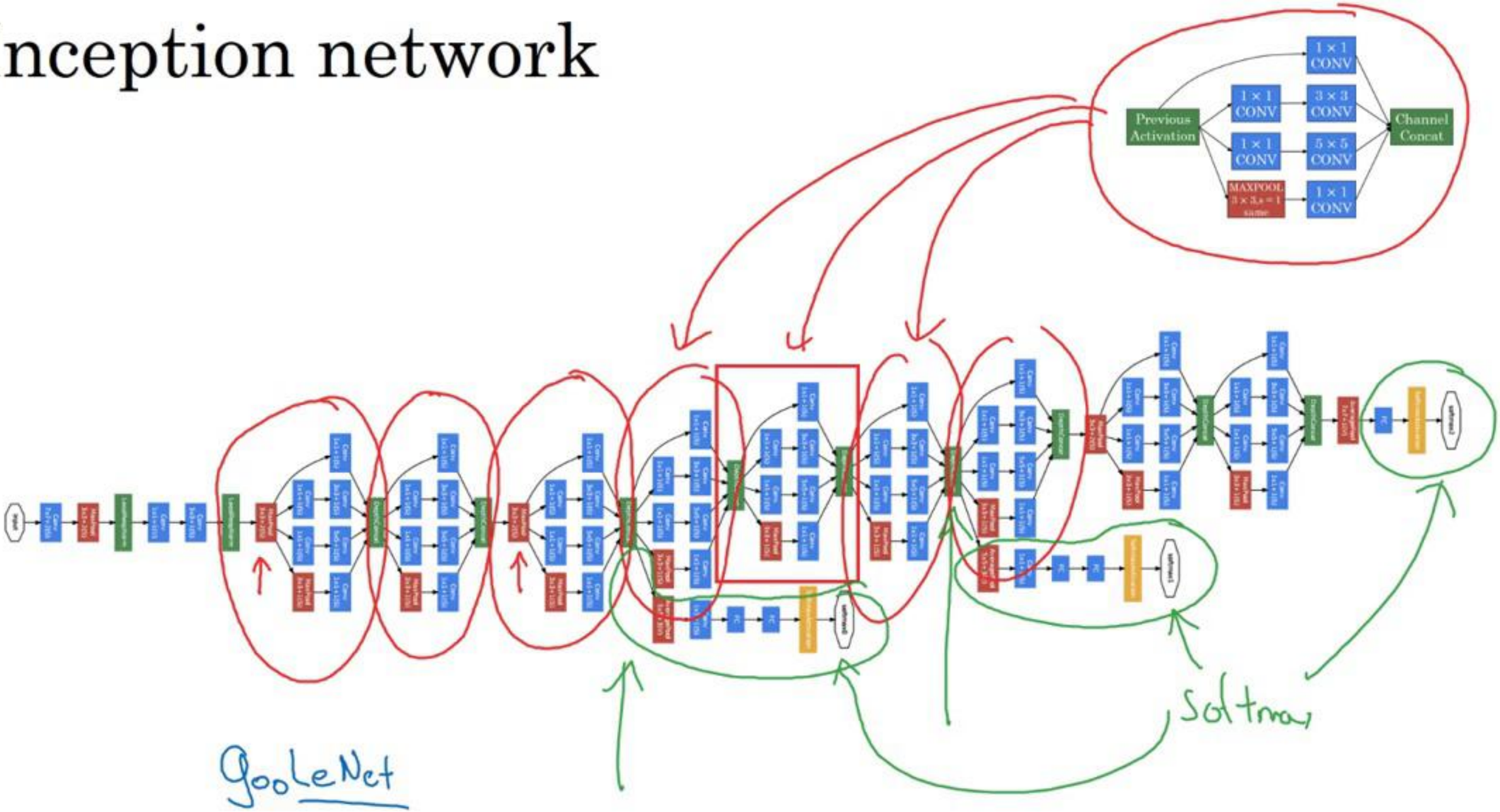




[Szegedy et al., 2014, Going Deeper with Convolutions]

Andrew Ng

Inception network



[Szegedy et al., 2014, Going Deeper with Convolutions]

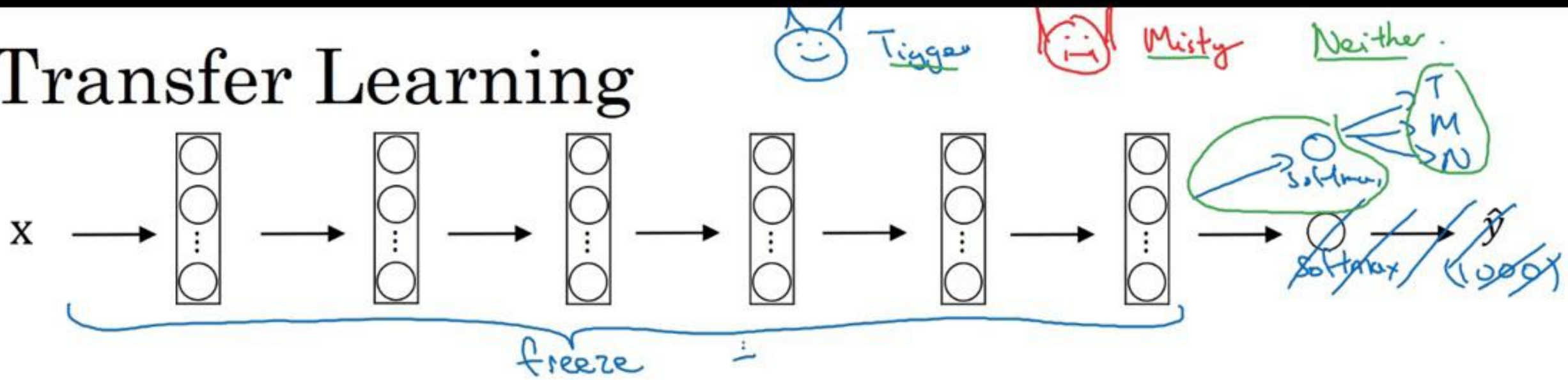
Andrew Ng

Practical advice for using ConvNets

deeplearning.ai

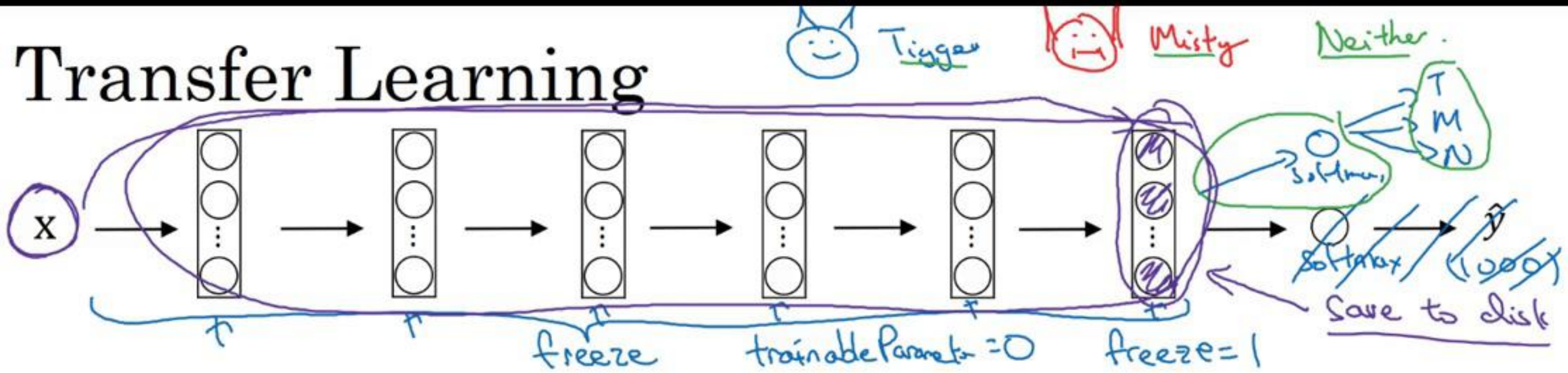
Transfer Learning

Transfer Learning



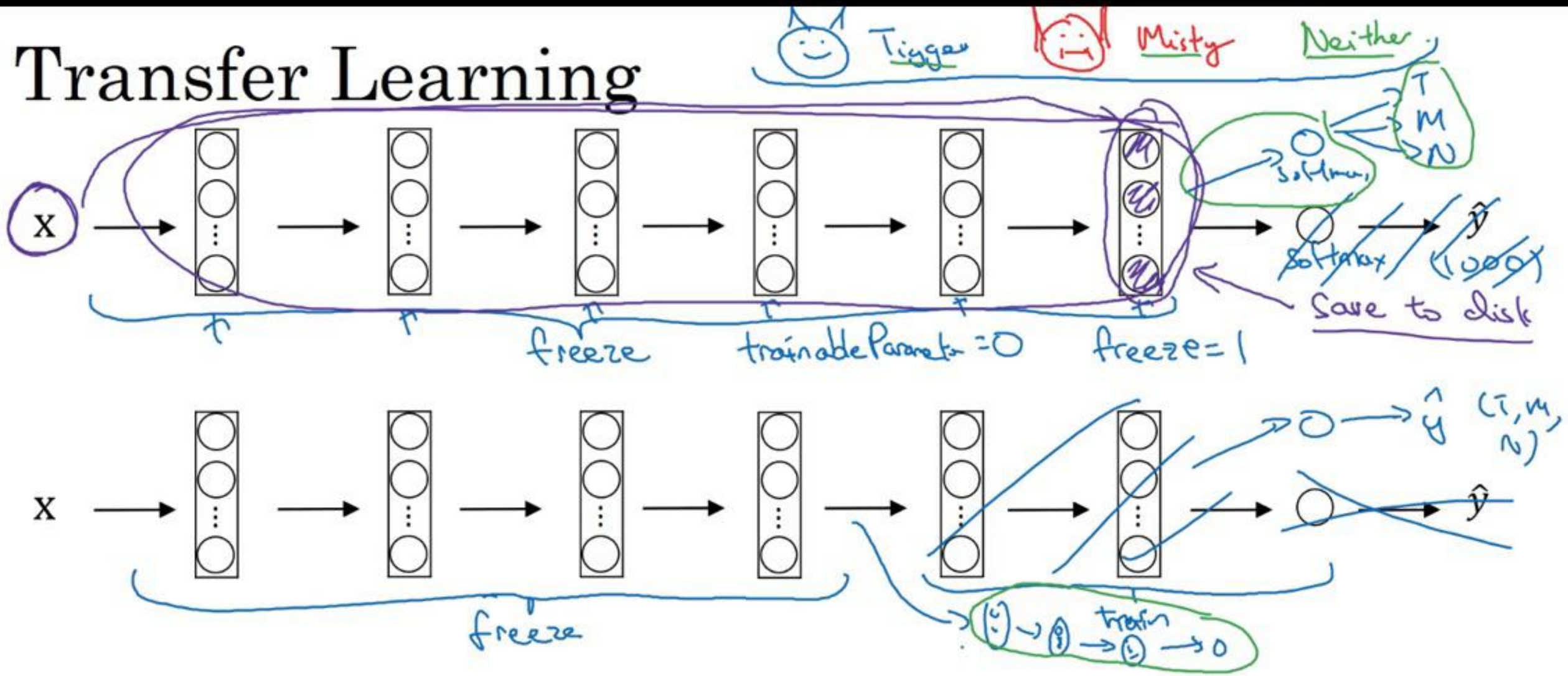
Andrew Ng

Transfer Learning



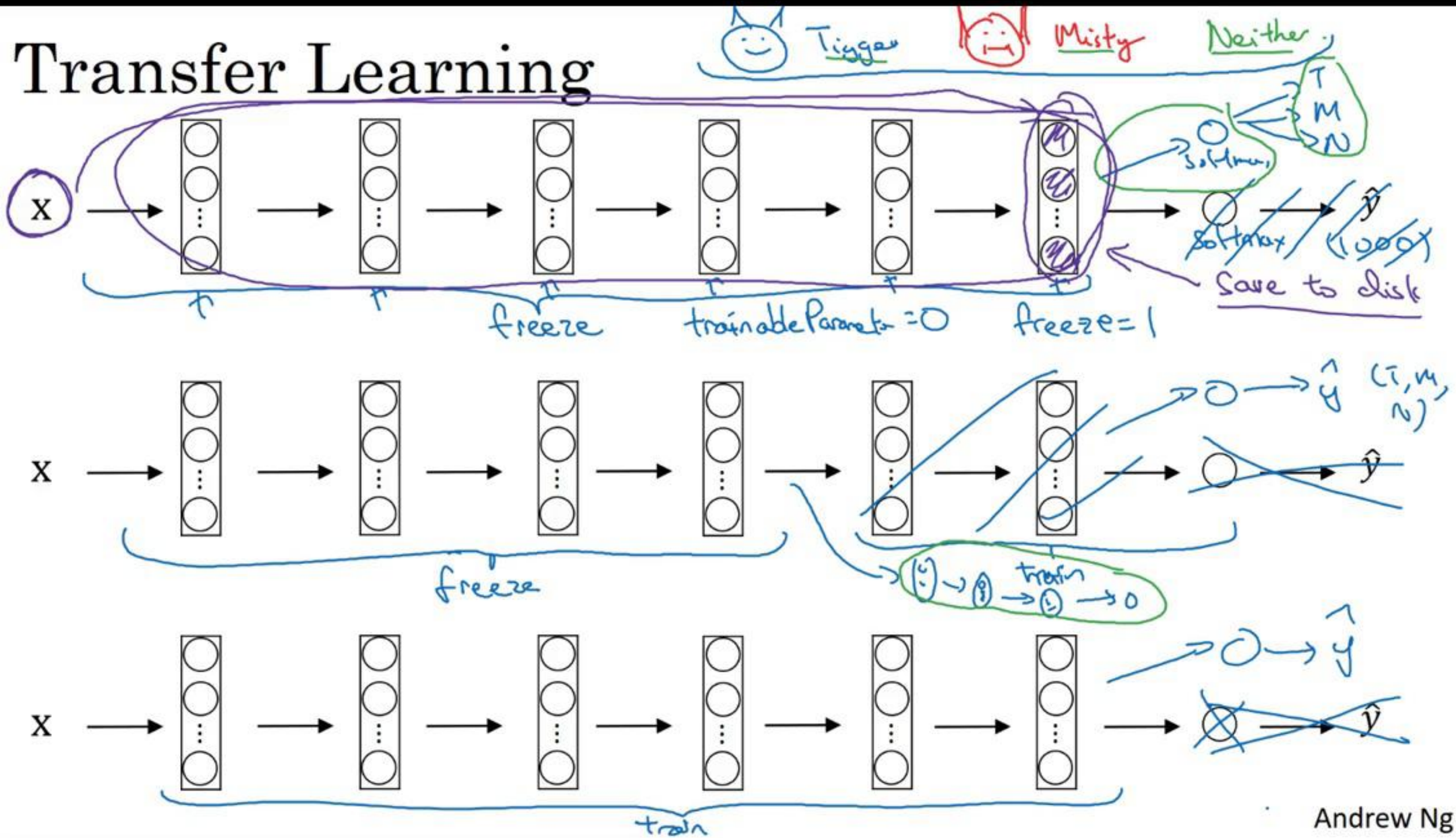
Andrew Ng

Transfer Learning



Andrew Ng

Transfer Learning



Andrew Ng

Practical advice for using ConvNets

deeplearning.ai

Data augmentation

Common augmentation method

Mirroring

y

Random Cropping

Andrew Ng

Common augmentation method

Mirroring

y

Random Cropping

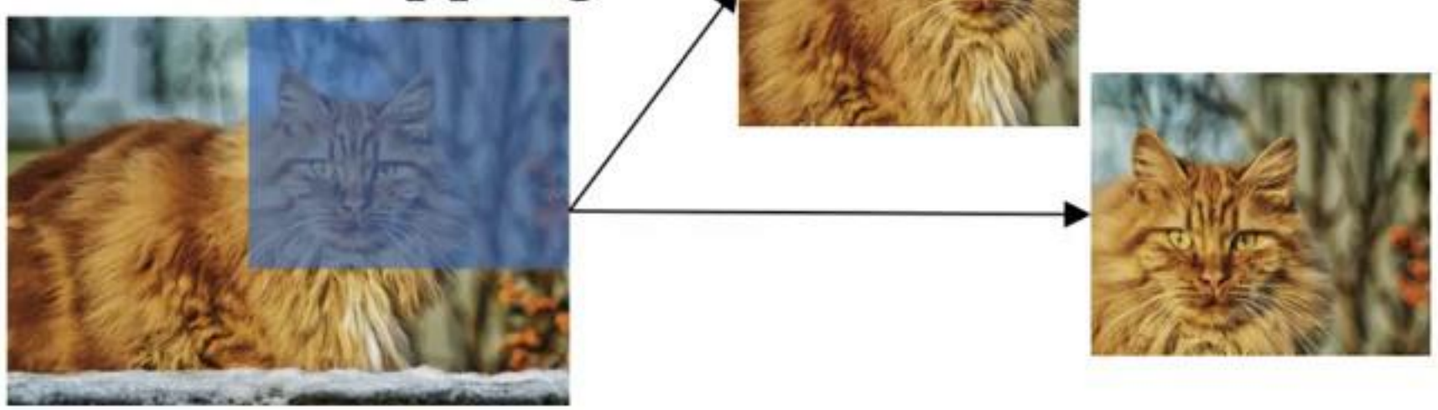
Andrew Ng

Common augmentation method

Mirroring

y

Random Cropping



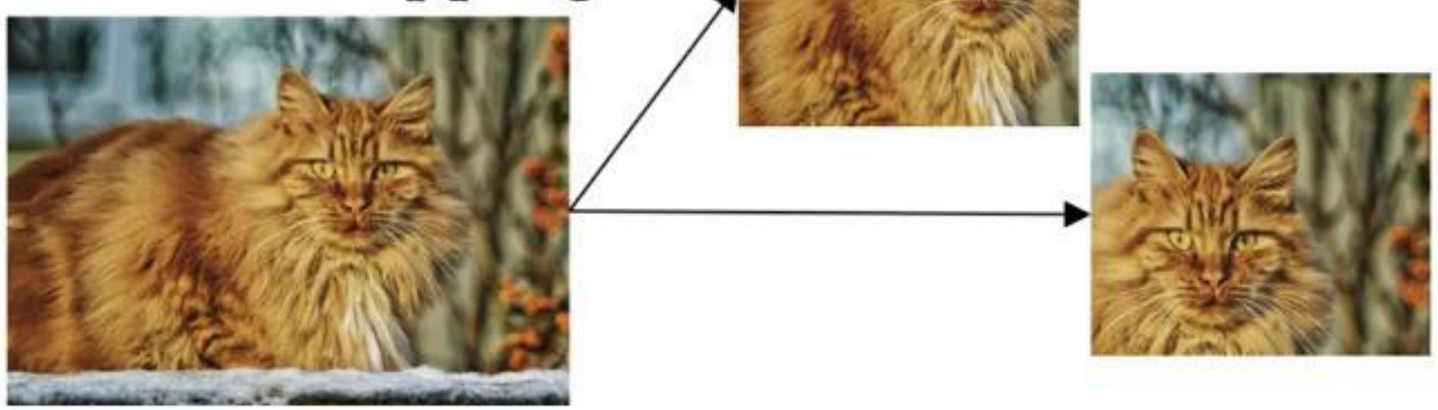
Andrew Ng

Common augmentation method

Mirroring

y

Random Cropping



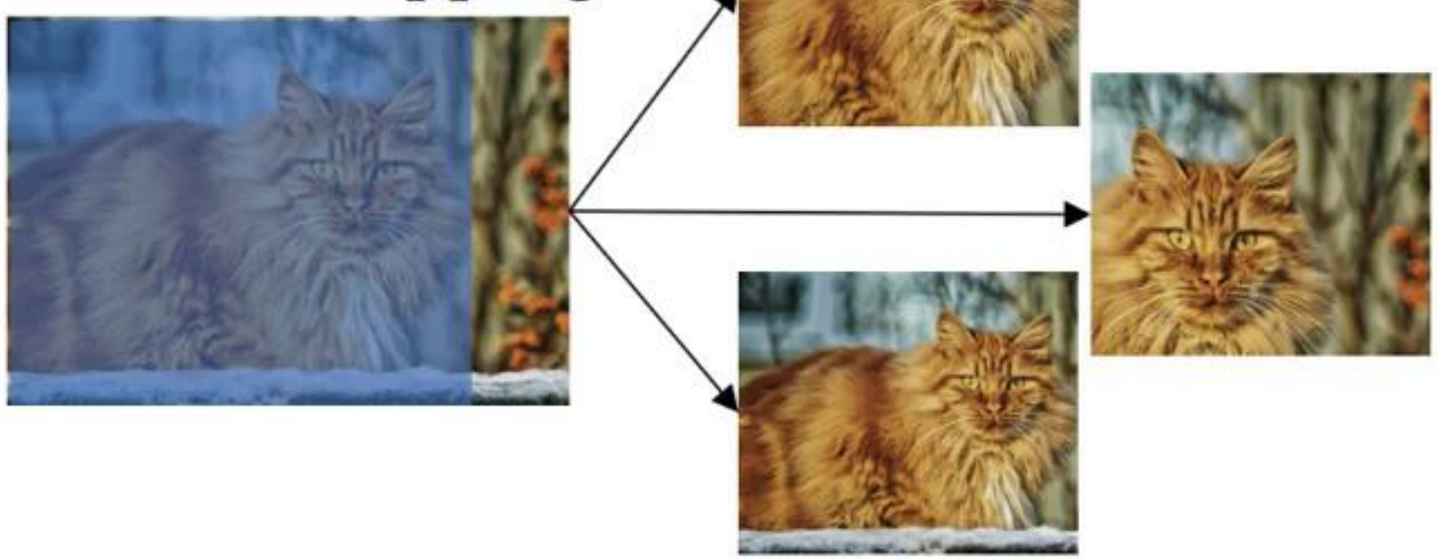
Andrew Ng

Common augmentation method

Mirroring

y

Random Cropping



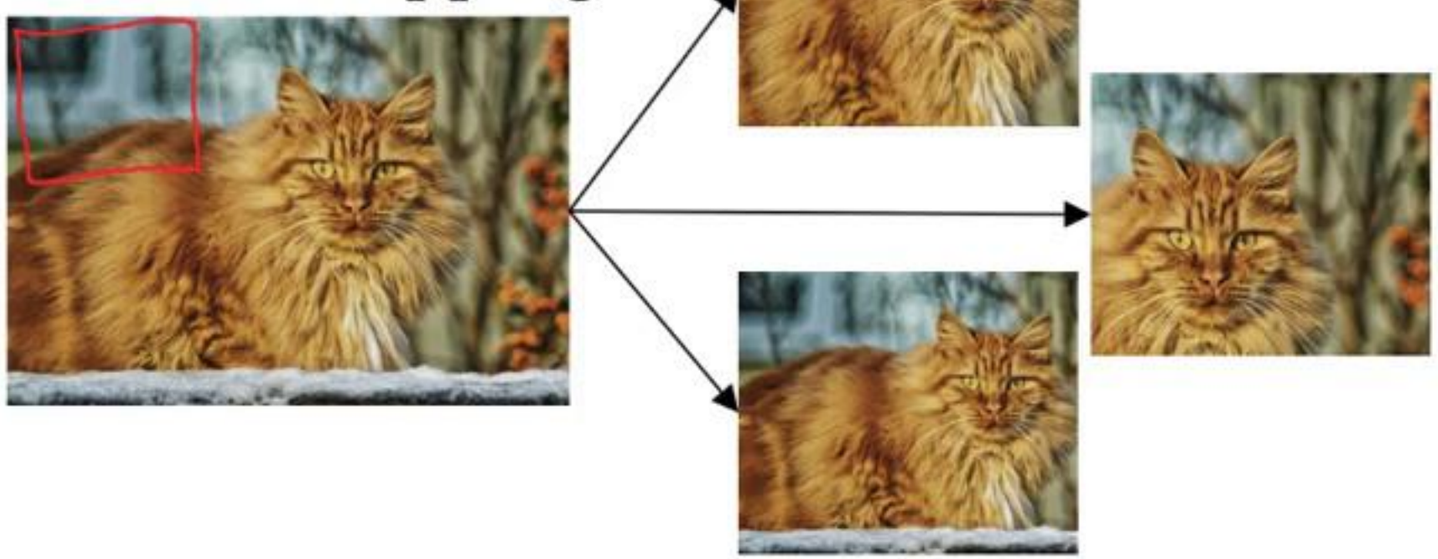
Andrew Ng

Common augmentation method

Mirroring

y

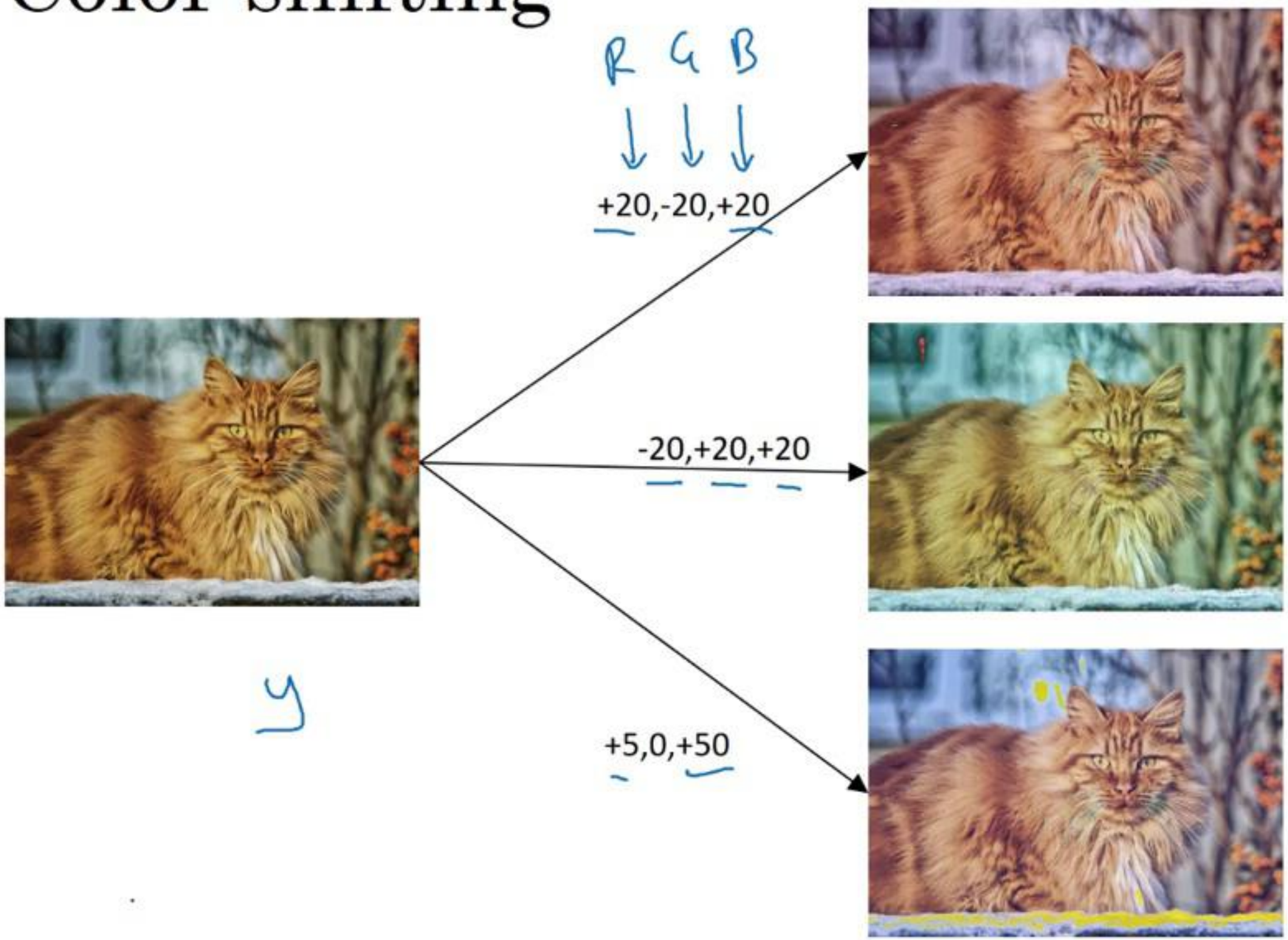
Random Cropping



- Rotation
- Shearing
- Local warping
- ...

Andrew Ng

Color shifting



Advanced:

PCA

ml-class.org

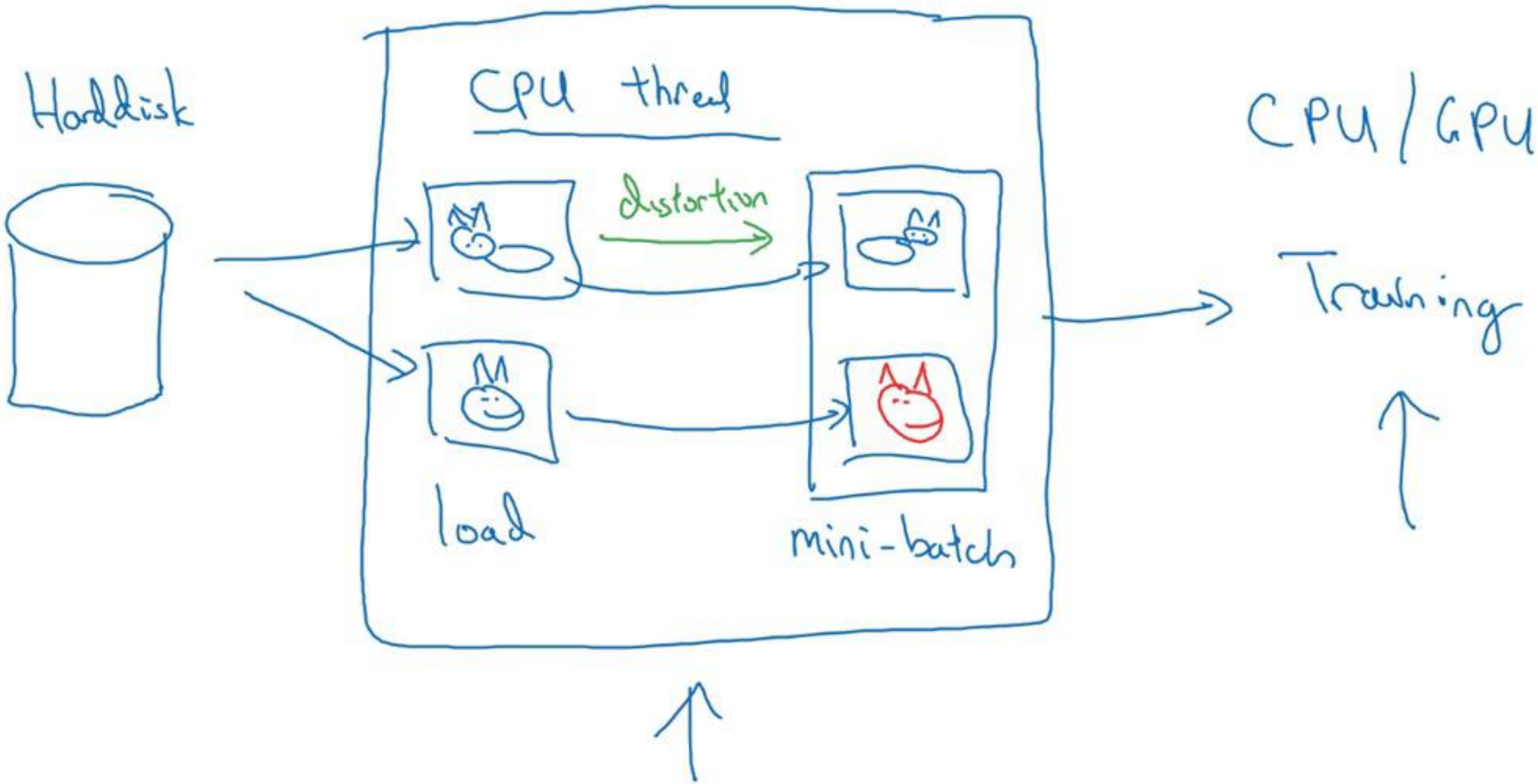
[AlexNet paper

["PCA color augmentation."

R B G

Andrew Ng

Implementing distortions during training



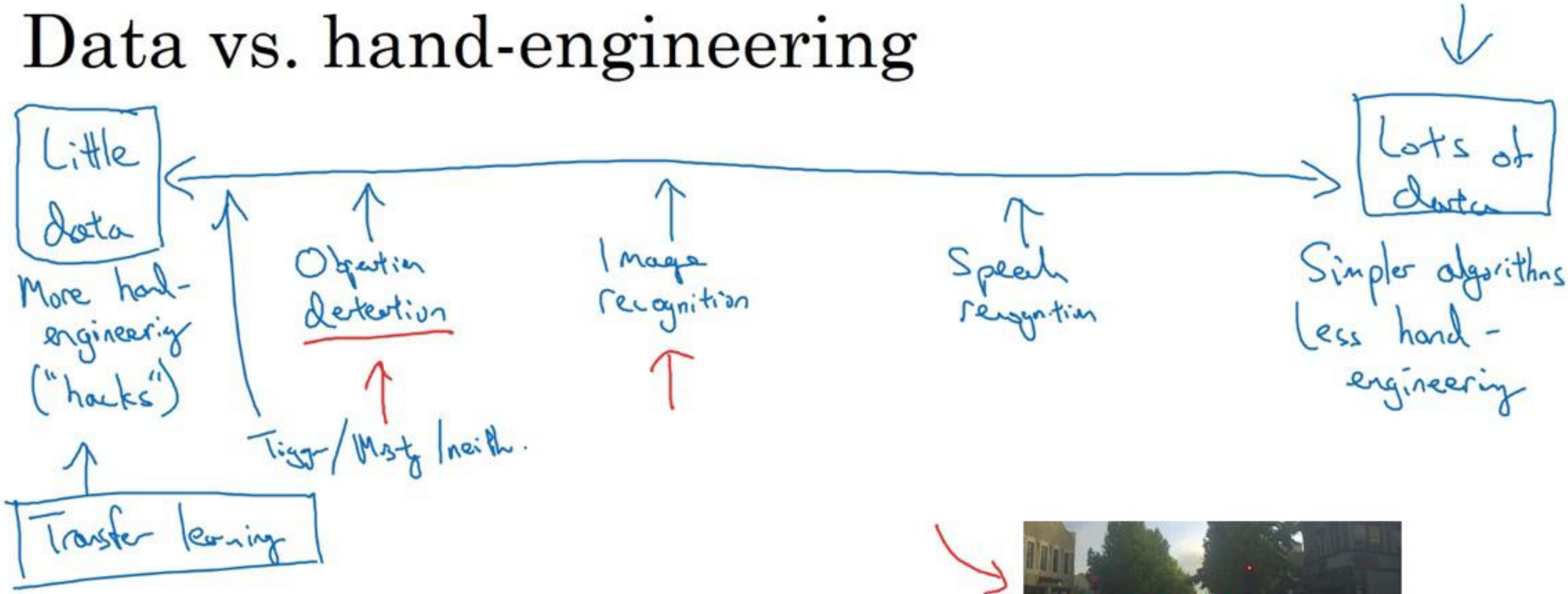
Andrew Ng

Practical advice for using ConvNets

deeplearning.ai

The state of computer vision

Data vs. hand-engineering



Two sources of knowledge

- Labeled data (x, y)
- Hand engineered features/network architecture/other components

Andrew Ng

Tips for doing well on benchmarks/winning competitions

Ensembling

3-15 networks

$\rightarrow \hat{y}$

- Train several networks independently and average their outputs

Multi-crop at test time

- Run classifier on multiple versions of test images and average results

10-crop

Andrew Ng

Tips for doing well on benchmarks/winning competitions

Ensembling

3-15 networks

$\rightarrow \hat{y}$

- Train several networks independently and average their outputs

Multi-crop at test time

- Run classifier on multiple versions of test images and average results

10-crop

Andrew Ng

Tips for doing well on benchmarks/winning competitions

Ensembling

3-15 networks

$\rightarrow \hat{y}$

- Train several networks independently and average their outputs

Multi-crop at test time

- Run classifier on multiple versions of test images and average results

10-crop

Andrew Ng

Tips for doing well on benchmarks/winning competitions

Ensembling

3-15 networks

$\rightarrow \hat{y}$

- Train several networks independently and average their outputs

Multi-crop at test time

- Run classifier on multiple versions of test images and average results

10-crop

Andrew Ng

Tips for doing well on benchmarks/winning competitions

Ensembling

3-15 networks

$\rightarrow \hat{y}$

- Train several networks independently and average their outputs

Multi-crop at test time

- Run classifier on multiple versions of test images and average results

10-crop

Andrew Ng

Tips for doing well on benchmarks/winning competitions

Ensembling

3-15 networks

$\rightarrow \hat{y}$

- Train several networks independently and average their outputs

Multi-crop at test time

- Run classifier on multiple versions of test images and average results

10-crop

1 + 4 + 1 + 4

Andrew Ng

Use open source code

- Use architectures of networks published in the literature
- Use open source implementations if possible
- Use pretrained models and fine-tune on your dataset